

Mini-BEHAVIOR-Gran: Revealing U-Shaped Effects of Instruction Granularity on Language-Guided Embodied Agents

Anonymous ACL submission

Abstract

Instruction granularity is an important yet poorly controlled variable in language-guided embodied AI. Existing benchmarks typically pair each task with a single static instruction, making it difficult to study how agent behavior changes when the same task is described at different levels of detail. We introduce MINI-BEHAVIOR-GRAN, a new benchmark for controlled studies of instruction granularity that extends MINI-BEHAVIOR with multiple instruction variants per task, ranging from high-level goal descriptions to step-by-step guidance. Using this benchmark, we compare four candidate metrics for cross-task granularity quantification: token count, entity count, action-verb count, and *planning-width*, and find that width correlates most consistently with agent performance. Using width to organize training and evaluation further reveals a non-monotonic U-shaped relationship between instruction granularity and performance, with peaks at both fine and coarse extremes. Further analysis suggests that the coarse-granularity performance rebound is associated with shallow grounding, where agents learn vision-dominant policies.

1 Introduction

Language-guided embodied agents learn a policy $\pi(a|v, l)$ that maps visual observations v and language instructions l to executable actions a . Recent progress has largely focused on model architectures, most notably the Vision-Language-Action (VLA) paradigm (Kim et al., 2024; Black et al., 2025; Shukor et al., 2025) and its advanced variants that adapt *world models* for future frame prediction ($v_t \mapsto v_{t+1}$) (Wu et al., 2026; Zhao et al., 2025; Bi et al., 2025) or *Chain-of-Thought* (CoT) reasoning for intermediate planning (Ye et al., 2025; NVIDIA et al., 2025; Yin et al., 2025). Yet the impact of *instruction granularity* remains underexplored. Instruction granularity refers to the level of detail provided in l to guide agent decision-making (Aru-

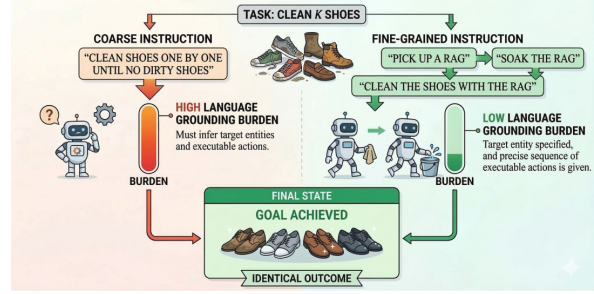


Figure 1: For the same task (cleaning k shoes), a coarse instruction requires the agent to infer target entities and executable actions. In contrast, a fine-grained instruction provides more detailed steps, significantly reducing the language grounding effort to reach the goal.

mugam et al., 2017), and it is typically an uncontrolled variable in current datasets and evaluations. It matters because varying granularity fundamentally alters the cross-modal alignment difficulty and the latent planning burden imposed on the agent. Understanding this impact is crucial not only for optimizing instruction design during training, but also for ensuring model robustness across diverse human instruction styles encountered at test time.

A systematic investigation of instruction granularity has been hindered by two key bottlenecks. First, existing embodied AI benchmarks lack controlled variation in instruction granularity. Typically, each task instance is paired with a single, static instruction, making it difficult to study how an agent’s performance scales as instructions shift from fine-grained (e.g., detailed step-by-step) to coarse (e.g., high-level goal descriptions). For example, popular benchmarks such as LIBERO and BEHAVIOR-1K provide only fixed or high-level instructions. Even recent suites like LIBERO-PLUS (Fei et al., 2025), which aim to increase diversity, primarily vary visual conditions (e.g., lighting, sensor noise) while keeping the language input fixed.

Second, and perhaps more fundamentally, the

field lacks a well-accepted and formalized metric for quantifying *instruction granularity*. Since granularity represents the density of guidance provided for decision-making, an ideal metric should directly reflect the complexity of grounding language into executable actions. With the advent of Large Language Models (LLMs), a seemingly natural proxy is to use the hierarchical depth of an instruction, i.e., its specific level within a decomposed plan, as a measure of granularity (Gao et al., 2025). However, such a metric fails to support meaningful cross-task comparisons because it conflates instructional detail with the task’s inherent complexity. Different tasks exhibit fundamentally different internal structures: some involve interleaved subgoals that resist clean hierarchical decomposition (e.g., the Sussman anomaly; Sussman, 1975), whereas others can be arbitrarily elongated into superficial substeps. Consequently, hierarchical levels cannot establish a consistent granularity scale across diverse tasks, thereby undermining their utility for investigating the general impacts of granularity on language grounding.

To enable systematic study of how instruction granularity shapes language-guided embodied agents, we introduce MINI-BEHAVIOR-GRAN, an extension of MINI-BEHAVIOR (Jin et al., 2023) that provides multiple instruction variants per task, spanning from high-level goal description to step-by-step instructions. Leveraging Minigrad’s engine (Chevalier-Boisvert et al., 2023), we construct 803 unique problem instances across 20 domains, yielding 10,253 trajectory episodes, each paired with dynamic instructions that evolve as the agent progresses through the task. Within this controlled setting, we compare four metrics for quantifying granularity: token count, entity count, action-verb count, and *width*. Adapted from width-based planning, *width* measures planning complexity as the maximum number of state features that must be jointly tracked to execute an instruction (Bonet et al., 2019; Drexler et al., 2024). As discussed earlier, while *decomposition depth* might seem a natural proxy for latent planning complexity, it fails to support cross-task comparison. In contrast, *width* is defined over a *shared feature space* common to all tasks, enabling scores that are comparable across tasks and capture the inherent planning burden an instruction imposes.

Our study shows that *width* correlates more consistently with agent performance across task horizons and VLA model variants than the other three

metrics, suggesting that it is a stronger quantitative proxy for instruction granularity in language grounding. Using *width* to organize training conditions at different granularities, we further uncover a non-monotonic U-shaped relationship between instruction granularity and agent performance. Finally, we probe the mechanisms underlying this effect. Using instruction predictability ($P(l|v)$) as a diagnostic signal, we find that the performance rebound at coarse granularity is associated with shallow grounding, whereby agents tend to bypass multimodal alignment and rely on vision-dominant policies.

Contribution. We make the following contributions: (1) Introduce MINI-BEHAVIOR-GRAN, a benchmark for controlled study of instruction granularity. (2) Demonstrate that *width* is the most consistent cross-task proxy among the metrics we evaluate. (3) Unveil a non-monotonic U-shaped relationship between granularity and agent performance; and (4) Provide evidence that the coarse-granularity performance rebound is linked to shallow grounding.

2 MINI-BEHAVIOR-GRAN: A Controlled Benchmark for Instruction Granularity

Following prior work on varying granularities and abstraction in robot instructions (Arumugam et al., 2019), we define *instruction granularity as the level of details and abstraction with which a language instruction specifies a task. In embodied settings, instruction granularity determines the latent planning complexity that an agent must resolve to map instructions into executable actions.*

Reiterating the example in Figure 1, we note that a coarse instruction (e.g., “clean the shoes”) leaves the agent with a large language grounding gap, as it must infer the target entities (e.g., which shoes to clean) and the executable actions (e.g., how to clean). In contrast, a fine-grained instruction provides detailed steps that significantly reduce the language grounding effort to fulfill the same task. In this section, we describe how we construct MINI-BEHAVIOR-GRAN to systematically vary instruction granularity for the same task.

2.1 Shared Feature Space and Symbolic Layer

Many embodied AI simulators expose symbolic representations alongside raw pixel observations for task specification, progress monitoring, or success evaluation (Shridhar et al., 2020; Puig et al.,

2020; Li et al., 2022a,b). We leverage this symbolic backbone to define a shared feature space Φ common across all tasks, representing each environment state s as a feature vector $\phi(s) = (\phi_1(s), \dots, \phi_n(s))$. Features in Φ include both Boolean variables (e.g., H_r : holding the rag; S_r : the rag is soaked) and numerical variables (e.g., p_r : distance to the rag; N_f : number of uncleaned shoes). This shared feature space provides the underlying vocabulary for representing instructions, which in turn allows us to quantify instruction granularity across tasks.

2.2 Rule-Based Instruction Representation

Building on the shared feature space Φ , we construct each instruction as a set of “if-then” rules that specify desired state progression. This choice is motivated by the fact that sequential decision-making tasks in automated planning are commonly modeled in terms of state variables and transitions with preconditions and effects, following the STRIPS formalism (Haslum et al., 2019). Thus, rule-based instructions explicitly encode the latent planning complexity an instruction imposes, as agents must determine how to transition from states satisfying a rule’s precondition to those satisfying its effect. Concretely, an instruction in this study is a set of rules $\mathcal{R} = \{r_1, \dots, r_m\}$, where each rule r_i is of the form $C_{r_i} \mapsto E_{r_i}$:

- C_{r_i} (the *condition*) is a conjunction of feature constraints. For a Boolean feature p , a constraint is p or $\neg p$; for a numerical feature n , it is $n = 0$ or $n > 0$.
- E_{r_i} (the *effect*) is a conjunction of feature changes. For a Boolean feature p , an effect is p , $\neg p$, or $p?$ (allowing any change); for a numerical feature n , it is $n \downarrow$ (decrease), $n \uparrow$ (increase), or $n?$ (any change).

For each task, we begin by manually constructing a finest-grained rule set $\mathcal{R}^{(0)}$ that decomposes the task into atomic steps, i.e., each rule typically can be fulfilled by a single action. We use the task of *cleaning k shoes* as a running example. Table 1 lists the relevant features from the shared space Φ used in this task.

$\mathcal{R}^{(0)}$ contains over a dozen atomic rules. For brevity, we show representative rules from each phase (full set in § G).

Feature	Description
H_r	holding the rag (Boolean)
p_r	distance to the nearest rag (numerical)
p_s	distance to the sink (numerical)
p_f	distance to the nearest uncleaned shoe (numerical)
N_f	number of uncleaned shoes (numerical)
T_s	tap (sink) is turned on (Boolean)
S_r	rag is soaked (Boolean)
C_i	shoe i is cleaned (Boolean)
O_r	rag is on the floor (Boolean)

Table 1: Features used in the Cleaning Shoes task.

Representative rules from $\mathcal{R}^{(0)}$ (Cleaning Shoes).

$\{\neg H_r, p_r > 0\} \mapsto \{p_r \downarrow\}$	(approach rag)
$\{H_r, \neg T_s, p_s = 0\} \mapsto \{T_s\}$	(turn on tap)
$\{H_r, T_s, p_s = 0\} \mapsto \{S_r\}$	(soak rag)
$\{S_r, N_f > 0, p_f > 0\} \mapsto \{p_f \downarrow\}$	(approach shoe)
$\{S_r, N_f > 0, p_f = 0\} \mapsto \{N_f \downarrow, \neg H_r\}$	(clean and drop)
$\{N_f = 0, \neg O_r, H_r\} \mapsto \{\neg H_r, O_r\}$	(put away rag)

A coarse rule is not created by mechanically concatenating the preconditions and effects of its constituent atomic rules. Instead, we treat each merge as an abstraction of a contiguous subplan segment into a macro-rule. The macro-rule exposes only the *net input/output interface* of that segment: its condition specifies when the segment applies, and its effect captures the overall progress achieved. Features that can be consumed entirely within the segment are thus omitted from the coarse rule. As shown in Figure 2, the $\mathcal{R}^{(2)}$ rules for picking up the rag (r_{1a}), soaking it (r_{1b}), and cleaning one shoe (r_{1c}) form a coherent subplan chunk r_1 that abstracts into a single coarse rule in $\mathcal{R}^{(3)}$, which captures the net progress of cleaning one shoe while hiding the intermediate features H_r and S_r .

After each merge, we verify the semantic validity of the resulting coarse rules through expert review and use a planning solver to ensure every rule sequence $\mathcal{R}^{(k)}$ is well-formed: sequentially reachable, goal-terminating, and dead-end-free (Bonet and Geffner, 2021).

2.3 Dynamic Instruction Generation

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis,

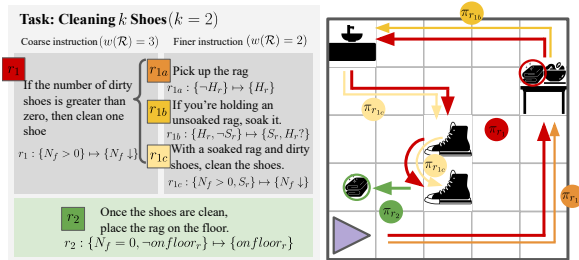


Figure 2: A running example of constructing coarse-grained instructions by merging atomic rules into macro-rules. Features H_r and S_r are omitted as they can be consumed entirely within the subplan chunk.

viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc ele-

mentum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

State Progression	Concurrent Features Tracked (Sequence of Tuples)	Width
Initial State	$[\langle \rangle]$	0
Navigate to rag	$[\langle p_r = X \rangle, \dots, \langle p_r = 0 \rangle]$	1
Pick up rag	$[\langle p_r = 0 \rangle, \langle H_r \rangle]$	1
Navigate to sink	$[\langle H_r, p_s = X \rangle, \dots, \langle H_r, p_s = 0 \rangle]$	2
Turn on tap (sink)	$[\langle H_r, p_s = 0 \rangle, \langle T_s \rangle]$	2
Soak rag	$[\langle H_r, T_s, p_s = 0 \rangle, \langle S_r \rangle]$	3
Navigate to Shoe 1	$[\langle H_r, S_r, p_{f1} \downarrow \rangle, \dots]$	3
Clean Shoe 1	$[\langle H_r, S_r, p_{f1} = 0 \rangle, \langle C_1 \rangle]$	3
Navigate to Shoe 2	$[\langle H_r, S_r, C_1, p_{f2} \downarrow \rangle, \dots]$	4
Clean Shoe 2	$[\langle H_r, S_r, C_1, p_{f2} = 0 \rangle, \langle C_1, C_2 \rangle]$	4
Put rag on floor	$[\langle C_1, C_2, H_r \rangle, \dots, \langle C_1, C_2, onfloor_r \rangle]$	3

Table 2: State progression and width for the “Cleaning k Shoes” ($k = 2$). Width scales as $w = k + 2$.

2.4 Benchmark Analysis

3 Quantifying Instruction Granularity

3.1 Desiderata for Cross-Task Granularity Metrics

3.2 Candidate Metrics

3.3 Instantiating Width in MINI-BEHAVIOR-GRAN

4 Experiments

4.1 Experimental Setup

4.2 RQ1: Which metric best captures instruction granularity?

4.3 RQ2: How does agent performance vary across instruction granularity?

4.4 RQ3: What drives the observed U-shaped effect?

4.5 RQ4: How robust is this effect across transfer and failure settings?

5 Rule Width as Instruction Granularity

5.1 Running Example

We first illustrate *planning width* computation using a running example. Table 2 shows the desired state progressions of a task where two shoes are cleaned using a rag, and the rag is placed on the floor afterwards. The key **environment dynamics** are: (1) the rag must be soaked before it can clean shoes, and (2) the tap (sink) must be turned on to soak the rag. We represent states using a set of task-relevant feature variables (Φ): (i) boolean features H_r (holding rag), T_s (tap (sink) is turned on), S_r (rag is soaked), C_i (shoe i is cleaned), $onfloor_r$ (rag is on the floor); and (ii) spatial features p_r, p_s, p_f (distance to the rag, sink, and nearest uncleaned shoe, respectively).

First, approaching the rag is a navigation-only segment: the desired state progression is to de-

crease the distance feature p_r until $p_r = 0$. Thus, the width is 1 as only a single feature needs to be tracked. In this environment dynamics, $p_r = 0$ enables the state transition to H_r . Thus, the rag can be obtained at $p_r = 0$ and the width remains 1. Subsequently, H_r must be maintained while progressing to next states, resulting in a width of 2. The rag becoming soaked requires simultaneous maintenance of three features, i.e., at the sink ($p_s = 0$), holding the rag (H_r), and having the tap (sink) turned on (T_s), which yields a width of 3. After soaking, C_1 can be obtained by tracking H_r , S_r , and p_f , giving width 3. The subtlety arises when cleaning the second shoe: one has to track the same features as before, plus maintaining the cleanliness of the first shoe (C_1), which increases the width to 4. A special case of $w = 0$ occurs for situations where the desired state progression can be achieved in zero or one step (Bonet and Geffner, 2024); this will appear in our experimental analysis.

Why no action model? Agents’ action sets constrain how state transitions are executed, but *width* focuses on state-level feature dependencies induced by the environment dynamics. Different agents (e.g., a rover vs. a drone) may have different action spaces to achieve the same feature change, so emphasizing state progressions promotes generalization of width-based concepts across different embodiments.

The impact of *instruction granularity* on language-guided embodied agent performance remains underexplored largely because the field lacks a **formal, quantifiable, cross-task** measure of granularity. Building on the limitations of heuristic metrics discussed in § 1, we formally equate *instruction granularity* with *latent planning demand*, i.e., the implicit decision-making burden required to ground text into executable actions. We operationalize this using *rule width* (w) (Drexler et al., 2024).

5.2 Symbolic Task Representation

While our agents will be trained on raw multimodal observations, we adopt a symbolic, relational state abstraction as our problem setting. Many embodied benchmarks provide such structured representations: object properties and relations are expressed as *fluents* to form symbolic state or goal conditions internally in their engine (Shridhar et al., 2020; Li et al., 2024). Formally, we consider a classical planning problem $P = \langle \mathcal{F}, \mathcal{A}, I, G \rangle$, where $\mathcal{F}$ is a

set of fluents, $\mathcal{A}$ a set of actions, I the initial state, and G the goal.

The key advantage of width is its task-independence. By defining a shared feature set $\Phi = \{\phi_1, \dots, \phi_n\}$ across the domain, instructions can be formalized as a set of “if-then” rules specifying state progressions in terms of feature valuation vectors $\phi(s) = (\phi_1(s), \dots, \phi_n(s))$. This establishes a unified metric space where the level of granularity of any instruction can be compared across different tasks.

5.3 Instructions as Rule Sets $\mathcal{R}$

Concretely, we represent each instruction as a set of rules $\mathcal{R} = \{r_1, \dots, r_m\}$ defined over Φ , where each r_i is of the form $C_{r_i} \mapsto E_{r_i}$:

- C_{r_i} (the *condition*) is a conjunction of feature constraints. For a Boolean feature p , a constraint is p or $\neg p$; for a numerical feature n , it is $n = 0$ or $n > 0$.
- E_{r_i} (the *effect*) is a conjunction of feature changes. For a Boolean feature p , an effect is p , $\neg p$, or $p?$ (allowing any change); for a numerical feature n , it is $n \downarrow$ (decrease), $n \uparrow$ (increase), or $n?$ (any change).

These forms capture the core semantics of the instructions we study. A rule r_i is **fulfilled** by a state sequence $(s, \dots, s')$ if $\phi(s) \models C_{r_i}$ and the pair $(\phi(s), \phi(s'))$ satisfies the changes in E_{r_i} (e.g., $\forall (n \downarrow) \in E_{r_i}, \phi_n(s') < \phi_n(s)$) while keeping unmentioned features constant (i.e., $\phi_j(s') = \phi_j(s)$ for $j \notin E_{r_i}$). All instruction rule sets $\mathcal{R}$ are constructed to be executable and dead-end-free by satisfying the soundness and closure properties (i.e., they terminate at the goal) (Bonet and Geffner, 2021). This guarantee ensures that the agent’s performance strictly reflects the language grounding complexity of different granularity levels, isolating it from task unsolvability.

5.4 Rule Width as a Granularity Measure

The width $w(r_i)$ of a rule r_i represents the maximum number of features that must be simultaneously tracked in any state along an optimal transition from a state satisfying C_{r_i} to one satisfying E_{r_i} (Lipovetzky and Geffner, 2012). This is distinct from simply counting the features that appear in the condition or effect, as those may be handled sequentially; it reflects the maximal concurrency required to execute the rule. We define the granularity

of an instruction $\mathcal{R}$ as the maximum width among its constituent rules: $w(\mathcal{R}) = \max_{r_i \in \mathcal{R}} w(r_i)$.

Low width corresponds to fine-grained instructions that decompose tasks into minimal, easily grounded subproblems; high width indicates coarse instructions that leave a large “grounding gap” for the agent to resolve internally.

We continue the running example to demonstrate two instruction sets, each representing a different width level, with two variants visualized in Figure 2. We also append a **counting feature** in Φ to track the number of uncleaned shoes, denoted as N_f .

Coarse-Grained Instruction ($w(\mathcal{R}) = 3$) This instruction states only the high-level rules and their desired ordering, leaving the internal structure of each sub-task implicit:

- $r_1 : \{N_f > 0\} \mapsto \{N_f \downarrow\}$ (clean a shoe; $w = 3$)
- $r_2 : \{N_f = 0, \neg O_r\} \mapsto \{O_r\}$ (put the rag on the floor after cleaning shoes; $w = 1$)

Middle-Grained Instruction ($w(\mathcal{R}) = 2$) This variant decomposes r_1 to explicitly state the *soaking* process:

- $r_{1a} : \{\neg H_r\} \mapsto \{H_r\}$ (pick up the rag; $w = 1$)
- $r_{1b} : \{H_r, \neg S_r\} \mapsto \{S_r, H_r?\}$ (soak the rag)
- $r_{1c} : \{N_f > 0, S_r\} \mapsto \{N_f \downarrow\}$ (clean; $w = 2$)
- $r_2 : \{N_f = 0, \neg O_r\} \mapsto \{O_r\}$ (put the rag on the floor; $w = 1$)

This rule set is well-formed: it is goal-terminating via r_2 , and the rules are sequentially reachable, meaning the successor state of any fulfilled rule (that is not the goal) also meets the condition of at least one rule in $\mathcal{R}$. The set is also robust to perturbations: if an agent drops the rag after soaking it ($\neg H_r$), the rule set remains applicable by reverting to r_{1a} to reacquire the rag before proceeding to the cleaning task (r_{1c}).

Width as a Planning Proxy. Decomposing a task into a rule set $\mathcal{R}$ provides structural scaffolding, guiding the agent through intermediate “stepping stone” states. By design, we assume the execution of r_i does not negate established fluents unless explicitly required by E_{r_i} . This is the essence of language-guided decomposition. Without this assumption, language guidance would offer no reduction in complexity; the agent would need to track every feature since the initial state.

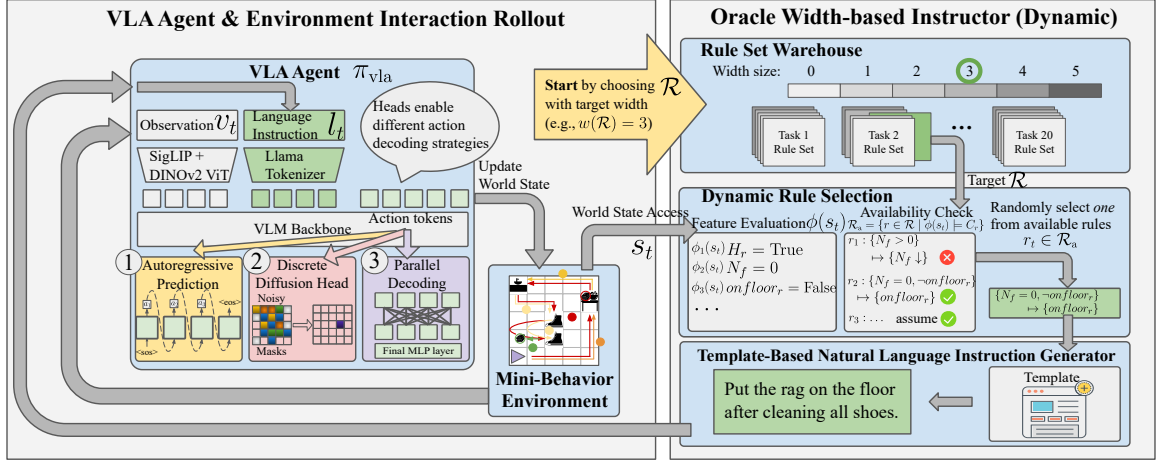


Figure 3: MINI-BEHAVIOR-GRAN Experimental Pipeline. We evaluate the impact of instruction granularity across three VLA action-decoding strategies. During inference time, an Oracle Instructor evaluates the current world state to decide available rule set $\mathcal{R}_a$ and active rule r_t . A rule-based natural language instruction l_t is generated accordingly. The process iterates until completion or max steps.

This intuition is formally grounded in novelty-driven Breadth-First Search (Lipovetzky and Geffner, 2017). By pruning states that fail to achieve a new feature conjunction of size $\leq k$, search complexity is bounded by $O(|\Phi|^k)$. Thus, trajectories that unnecessarily negate an already-achieved precondition C_{r_i} (unless required by E_{r_i}) are longer than those preserving it and are pruned from the optimal frontier. Consequently, features in C_{r_i} that are already true are effectively treated as invariants, and the active coordination w reduces to the incremental features needed for the next subgoal. For instance, while the monolithic task requires coordinating $\{H_r, p_s, T_s\}$ ($w = 3$, Table 3), the instruction r_{1b} leverages the fact that H_r is pre-established. By treating H_r as a fixed invariant, the search width w reduces to the novel features $\{p_s, T_s\}$, yielding $w = 2$.

To communicate these rules to language-guided embodied agents, we translate each rule into a natural-language sentence via a template-based generator. These templates verbalize the features within each rule; e.g., $\{H_r, \neg S_r\} \mapsto \{S_r\}$ is realized as “If you are holding the rag but the rag is not yet soaked, soak the rag.” This construction preserves the granularity differences defined by the rule width. Additional language prompt examples are provided in § C.

6 Mini-BEHAVIOR-Gran Benchmark

To disentangle the influence of instruction granularity in agent performance, we seek an environ-

ment that (1) retains long-horizon and heterogeneous activities so that instruction guidance meaningfully modulates the planning demand, while (2) simplifies perception and low-level control to avoid confounding factors. As such, we use MINI-BEHAVIOR (Jin et al., 2023) as our testbed. Detailed justification for our choice over other environments is provided in § C.

We introduce MINI-BEHAVIOR-GRAN, the first benchmark enabling controlled study of instruction granularity via *rule width*. Over 20 tasks sharing a feature set Φ , we design rule sets at up to six width levels, all verified solvable. During inference, an oracle instructor monitors the current feature vector $\phi(s_t)$, selects an available rule r_t from the task’s rule set, and translates it via a template into a natural language instruction l_t . This guarantees that performance differences reflect grounding ability alone, not stochastic instruction quality (see Figure 3). The instruction l_t and current observation v_t are then fed to the learning-based agent to predict action chunks $(a_t, \dots, a_{t+k}) \sim \pi_{\text{agent}}(\cdot \mid v_t, l_t)$.

7 Experiments

7.1 Experimental Setup

Task Suite & Training. We use MINI-BEHAVIOR-GRAN, which contains 20 complex tasks sharing the same feature set Φ . We further classify instruction granularity into three *relative* classes of *Fine* (F), *Medium* (M), and *Coarse* (C) via per-task quantile binning. In most cases,

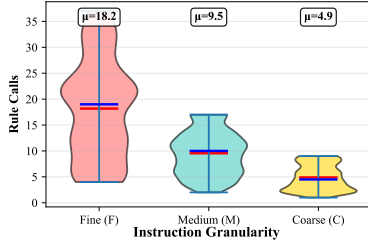


Figure 4: Rule call distribution across instruction granularities. # of rule calls decreases with coarser instructions. It decouples instruction length from width.

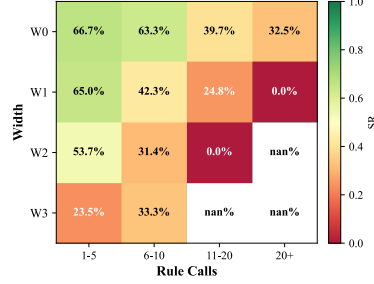


Figure 5: Impact of width and horizon on SR. SR decays with task horizon; larger instruction width imposes an additional burden even when horizon is fixed.

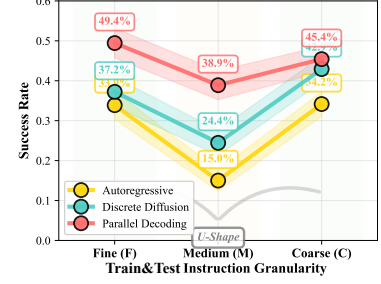


Figure 6: In-Distribution (matched train/test) SR across granularities (i.e., train Fine $\rightarrow$ test Fine, etc.). A non-monotonic U-shaped trend appears.

this mapping corresponds to $w = 0$ for Fine, $w = 1$ for Medium, and $w \geq 2$ for Coarse. We evaluate seven training settings: (1) **Pure F/M/C** (1 : 0 : 0): trains exclusively on one granularity class; (2) **Centroid** (1 : 1 : 1): uniformly sample from all three classes; (3) **Axial F/M/C** (3 : 1 : 1): emphasizes one class with some exposure to the other classes. Our dataset provides 50 training and 10 evaluation instances per task, each paired with reference trajectories generated by BFWS width-based planner from Lipovetzky and Geffner (2017) (see § F for details).

Action-Decoding Archetypes. The primary architectural difference among latest VLA models lies in their action-decoding strategies (Zhong et al., 2025). Hence, we employ a unified VLM backbone but vary three representative action-decoding strategies (see Figure 3 left panel): (1) **Autoregressive** (AR), exemplified by RT-2 (Zitkovich et al., 2023), OpenVLA (Kim et al., 2024) and others (Reed et al., 2022; Huang et al., 2024; O’Neill et al., 2024) that predict actions sequentially via causal attention; (2) **(Discrete) Diffusion**, which uses diffusion process for action chunk prediction (Black et al., 2025; Bjorck et al., 2025; Shukor et al., 2025). We employ discrete diffusion (Liang et al., 2025) for compatibility with the discrete action space of our testbed; (3) **Parallel Decoding**, utilizing bidirectional attention for single-pass action prediction to improve throughput, as demonstrated by OpenVLA-OFT (Kim et al., 2025b), PD-VLA (Song et al., 2025) and other variants (Yin et al., 2025; Wang et al., 2025; Xiao et al., 2025). Details on model hyperparameters, hardware and training schedules are in § H.

7.2 Establishing the Yardstick: Decoupling Horizon from Width

A critical confounder in evaluating language grounding complexity is the task sequence length. However, in language-guided execution, instructions inherently decompose long-horizon tasks into segments. Consequently, the relevant measure of sequential burden shifts from the raw atomic horizon (total action steps) to the instruction horizon, i.e., the number of rule calls issued by the instructor.

To isolate rule width from sequential burden, we train a single agent across all tasks using the Centroid configuration, ensuring balanced exposure across all granularity levels. This approach holds the distribution of atomic horizons constant, allowing us to isolate how instruction granularity and the frequency of rule calls affect performance. Figure 4 shows that fine-grained instructions ($w = 0$) decompose the task into numerous low-width rule calls; coarse instructions ($w \geq 2$), by contrast, yield fewer rule calls.

To assess whether w serves as an effective indicator of latent planning demand for learning-based agents, we analyze Figure 5. The heatmap reveals a strict monotonic decay in Success Rate (SR) as rule width increases. Also, for a fixed w , SR decays as rule calls increase, mirroring greater sequential horizon complexity. This confirms w as a horizon-independent measure of the intrinsic planning demand imposed by instruction granularity.

Width vs. Syntactic Heuristics. We evaluate whether w captures language grounding complexity better than common linguistic heuristics: token count, entity count, and action verbs. Figure 9 shows w is the only metric that exhibits a statisti-

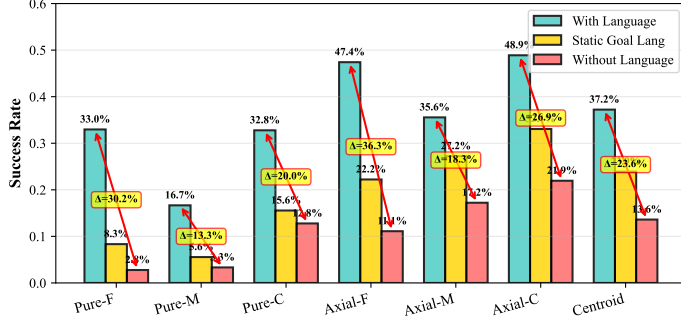


Figure 7: Smaller SR gaps indicate a shift towards shallow grounding. Under semantic perturbation, the Pure-C setting exhibits tiny variance (2.8%) between *w/ goal lang* and *w/o lang*.

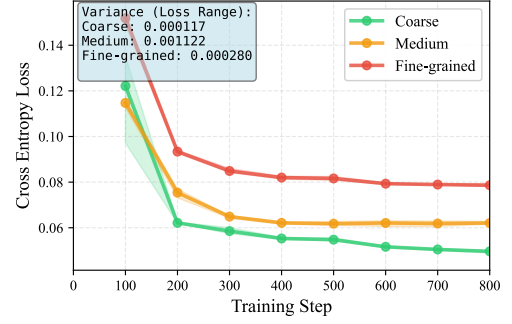


Figure 8: VLM instruction-prediction Cross Entropy (CE) loss across granularities and 3 seeds. Larger CE loss indicates lower $P(l | v)$.

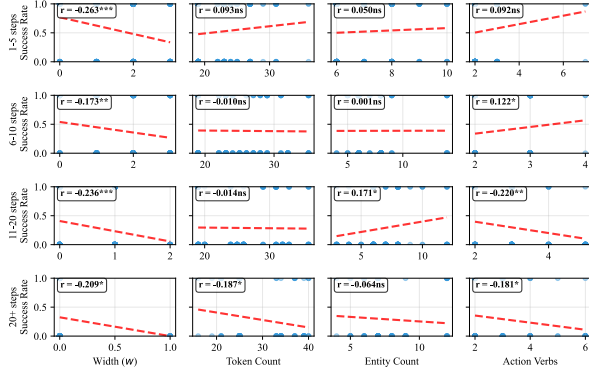


Figure 9: Metric-success correlation by task-horizon.

cally significant and consistently *negative* correlation with SR across task horizons. Action Verbs correlate significantly but flip direction across horizons, while Token Count turns negative only in the longest bin (20+ steps). These heuristics are inconsistent compared to w (see details in § I).

7.3 The U-Shaped Paradox and Capacity Cap

To establish a baseline, we train models using the *Pure F/M/C* setting. the models are trained and evaluated on the same granularity class. As shown in Figure 6, all VLA archetypes exhibit a non-monotonic *U-shaped performance curve*.

At the fine extreme ($w = 0$), where instructions require minimal latent reasoning to be grounded into actions, models achieve high performance. Performance then decreases at medium granularity ($w = 1$), before unexpectedly rebounding at the coarse regime ($w \geq 2$). This contradicts the assumption that higher planning width should monotonically degrade learning.

7.4 Mechanistic Probe: The Grounding Valley

We hypothesize that performance rebounds at $w \geq 2$ due to increasing instruction predictability

$P(l|v)$, which incentivizes agents to bypass language in favor of visuo-motor shortcuts.

Following prior work (Fei et al., 2025), we use $\Delta SR_{w/-w/o}$ as a measure of language grounding. To further investigate grounding effects, we conduct another intervention: replacing full instructions with high-level static goal descriptions (Figure 7). When language is entirely removed, fine-trained policies suffer a severe 36.3% drop, whereas coarse-trained policies show a smaller 20.0% gap. Also note that under reduced informativeness (*w/ goal lang*), the Pure-C policy exhibits a negligible 2.8% advantage over having no language at all (15.6% vs. 12.8%). This indicates that policies trained on coarse instructions develop a shallow grounding behavior: language is not deeply integrated into their decision-making. Consequently, when instructions are reduced to generic goals, their performance barely differs from the vision-only baseline.

The Predictability Driver: Why Reliance Shifts.

We hypothesize that this behavioral shift originates from coarse instructions being statistically more predictable from vision. Formally, consider a surjective mapping f from fine-grained labels L_{fine} to coarser ones L_{coarse} . The conditional probability of a coarse label aggregates its fine-grained constituents: $P(c_j|v) = \sum_{l_i \in f^{-1}(c_j)} P(l_i|v)$. The increasing predictability of coarse instructions reduces the marginal information contributed by language, making it easier for agents to minimize task loss by relying primarily on visual observations, thereby favoring the development of visuo-motor shortcuts over costly cross-modal alignment. We confirm this predictability empirically by training a VLM (Qwen3-VL-4B via LoRA) to act as an trained instructor, predicting instruc-

Training Setup	Width 0	Width 1	Width 2	Width 3	Width ≥ 4
Pure-F	34.9%	16.9%	10.8%	1.0%	0.0%
Pure-M	12.2%	22.0%	11.7%	0.0%	0.0%
Pure-C	26.3%	33.3%	32.9%	22.9%	0.0%
Axial-F	50.2%	53.3%	48.8%	34.3%	0.0%
Axial-M	44.7%	43.5%	45.8%	29.5%	0.0%
Axial-C	55.7%	52.2%	50.8%	37.1%	0.0%
Centroid (Mix)	42.7%	38.4%	40.8%	28.6%	0.0%

Table 3: The x-axis are test instruction width, y-axis are training setup. Transfer is asymmetric: train coarse $\rightarrow$ test fine works, but train fine $\rightarrow$ test coarse does not. Mixed-granularity improves overall.

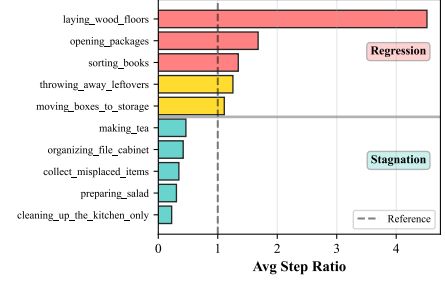


Figure 10: Low-width but long task horizon (e.g., laying woods) tends to cause regression, while high-width stagnates.

tions from visual observations. As Figure 8 shows, Cross-Entropy (CE) loss decreases monotonically with coarseness (Fine: 0.078, Medium: 0.063, Coarse: 0.047), a relative drop of 40% from Fine to Coarse. Together, these results form a complementary chain: probability aggregation explains *why* $P(l|v)$ increases, CE loss empirically confirms the increase, and the perturbation ΔSR quantifies the resulting visuo-motor shift.

Thus, we attribute this U-shaped behavior to how the agent dynamically shifts its reliance on text versus vision across different rule widths:

- **Fine: Deep Grounding.** Low $P(l|v)$ (high CE loss, Figure 8) means language provides unique information. Combined with low-width grounding burden (§ 7.2), the model reliably maps instructions to actions, yielding high SR (Figure 6).
- **Medium: The Alignment Trap.** At this width, grounding complexity increases while $P(l|v)$ remains moderate: not yet high enough to exploit reliable visuo-motor shortcuts. The model must attempt language grounding but struggles with its inherent difficulty. This intermediate regime, caught between hard alignment and insufficient shortcuts, yields the lowest performance.
- **Coarse: Reduced Language Reliance.** High $P(l|v)$ incentivizes a visual-dominant policy: aligning vision to action is easier than learning cross-modal grounding. This yields high in-distribution scores but at the cost of language grounding capability.
- **Capacity Cap ($w \geq 4$): Cognitive Overload.** Mirroring the $O(|\Phi|^4)$ search complexity of symbolic planners (Lipovetzky and Geffner, 2017), the performance of language-guided embodied agents across all training setups uniformly collapses at $w \geq 4$ (Table 3). This convergence

suggests that, within the scope of purely neural, end-to-end policies without explicit memory or search (as tested across three action-decoding archetypes), $w \geq 4$ poses a consistent language grounding bottleneck.

Deepening Grounding via Axial Training. Having diagnosed this shallow reliance as a byproduct of high predictability, we hypothesize that increasing the diversity of the training language space L should artificially lower the average $P(l|v)$, forcing the agent to structurally re-engage with the text. Our mixed-granularity training confirms this mechanism. All mixed-granularity models exhibit substantially larger ΔSR gaps compared to their *Pure-X* counterparts. Among these, the fine-weighted mixture *Axial-F* emerges as the optimal structural compromise: it achieves an overall success rate, while maintaining a significantly larger ΔSR .

7.5 Other Results

Asymmetric Generalization. We observe an asymmetric transfer pattern (Table 3): coarse-trained models successfully zero-shot transfer to fine-grained instructions, but fine-trained models fail on coarse-grained ones. This asymmetry aligns with our ΔSR analysis: coarse training induces a visual-dominant policy, hence perturbing or refining the instruction yields minimal behavioral change. Conversely, fine training induces tight coupling with language, making the policy brittle when detailed guidance is withdrawn.

Failure Modes. Agent failure distributions bifurcate strictly with width (Figure 10). Low-width, long-horizon tasks predominantly suffer from *regression*: the agent violates previously satisfied fluents and cycles back to prior states, reflecting subgoal serialization failures. High-width tasks ($w \geq 3$), by contrast, exhibit *stagnation*: the agent

freezes, unable to resolve concurrent feature dependencies ($O(|\Phi|^w)$) required to initiate a valid state transition. This dichotomy mirrors the width-based complexity landscape (see § I for details).

8 Related Work

Instruction Abstraction and Granularity. In this work, *Instruction granularity* refers to the extent to which state and action constraints are explicitly specified versus left for the agent to infer when grounding language into actions. This is distinct from, yet often conflated with the concept of *linguistic abstraction*. The term *abstraction* is viewed as *representational compression*—using a compact description to stand for multiple instances or executions. This definition is common in prior work of computational thinking (Wing, 2010; Lachmy et al., 2022; Zheng et al., 2024). Importantly, granularity can vary independently of abstraction: “tidy the room” is coarse but non-abstract (it leaves the exact required actions underspecified, yet it lacks representational compression.), whereas “repeat wiping until clean” is relatively specific yet abstract (it prescribes an action but uses a loop condition to compress multiple executions).

While not framed as the term “granularity”, we acknowledge that a rich body of work has focused on augmenting language specificity and its impact, for example through coarse-to-fine semantic parsing (Dong and Lapata, 2018; Li et al., 2020). Other work generates compositional instructions via constraint dropout and sub-task recombination (Huang et al., 2025b), or construct counterfactual instructions to augment diversity (Glossop et al., 2025).

Varying instruction detail is often handled via a simple two-part decomposition: high-level plans are converted into sequences of low-level steps before being grounded into actions (Arumugam et al., 2017; Yang et al., 2024; Shi et al., 2025). Most recent Vision Language Action (VLA) works, however, operate with static language instructions (Ma et al., 2025; Li et al., 2025). Some models like InstructVLA and ThinkAct explore having chain-of-thought reasoning steps before action decoding (Yang et al., 2025; Huang et al., 2025a). These hierarchical definitions remain task-specific and qualitative, lacking a unified, cross-task metric to establish shared granularity levels across tasks. Consequently, quantitatively controlled studies on the formal dynamics of instruction granularity remain scarce in the embodied AI domain.

Language Grounding vs. Shortcuts. Prior work highlights a dichotomy between fragile language sensitivity (Akula et al., 2022) and visuo-motor shortcuts (Xing et al., 2025; Zhang et al., 2026). We reconcile this via rule width (w): low w exhibits low predictability ($P(l|v)$), enforcing deep language grounding. Conversely, $w \geq 2$ yields high $P(l|v)$: this redundancy induces a shift toward shallow grounding, where agents default to visual priors.

9 Conclusion

This work introduces rule width (w) as a cross-task formalism for instruction granularity and demonstrates that it correlates with language grounding difficulty in learning-based agents. We observed a non-monotonic relationship between granularity and performance, driven by a trade-off between instruction predictability ($P(l|v)$) and latent planning complexity. We demonstrate that mixed-granularity training balances language sensitivity without sacrificing the robust visual reasoning skills learned from coarse trajectories.

10 Limitations

Discrete Action Spaces. We evaluate our framework in environments with discrete action spaces, a common abstraction in language-conditioned embodied AI (e.g., ALFWorld-based agent (Xiong et al., 2025; Kim et al., 2025a)) and LLM-based agent systems (Qin et al., 2024; Patil et al., 2024), where high-level reasoning is decoupled from low-level execution via skill primitives or API calls. This choice is intentional: it isolates symbolic planning from continuous control noise, allowing us to cleanly attribute observed effects to instruction granularity. However, extending this granularity-aware framework to hybrid discrete-continuous or continuous action spaces remains essential for real-world robotics applications, where low-level motor commands interact with high-level linguistic abstractions.

On the Oracle Instructor Setup. A potential concern is that providing the active rule via an Oracle simplifies the task to single-step grounding. However, we argue that the **latent planning demand** w is an intrinsic property of the instruction-to-action mapping. Even when the “what to do” is provided, the agent must still coordinate w concurrent features to achieve the transition. Our findings that performance collapses at $w \geq 4$ despite Oracle

guidance (Table 3) demonstrates that the grounding gap is a fundamental bottleneck in the learning process.

References

Arjun R. Akula, Spandana Gella, Aishwarya Padmakumar, Mahdi Namazifar, Mohit Bansal, Jesse Thomason, and Dilek Hakkani-Tur. 2022. ALFRED-L: investigating the role of language for action learning in interactive visual environments. In *EMNLP*, pages 9369–9378. Association for Computational Linguistics.

Dilip Arumugam, Siddharth Karamcheti, Nakul Gopalan, Edward C. Williams, Mina Rhee, Lawson L. S. Wong, and Stefanie Tellex. 2019. Grounding natural language instructions to semantic goal representations for abstraction and generalization. *Auton. Robots*, 43(2):449–468.

Dilip Arumugam, Siddharth Karamcheti, Nakul Gopalan, Lawson LS Wong, and Stefanie Tellex. 2017. Accurately and efficiently interpreting human-robot instructions of varying granularities. *arXiv preprint arXiv:1704.06616*.

Hongzhe Bi, Hengkai Tan, Shenghao Xie, Zeyuan Wang, Shuhe Huang, Haitian Liu, Ruowen Zhao, Yao Feng, Chendong Xiang, Yinze Rong, Hongyan Zhao, Hanyu Liu, Zhizhong Su, Lei Ma, Hang Su, and Jun Zhu. 2025. *Motus: A unified latent action world model*. *Preprint*, arXiv:2512.13030.

Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, and 1 others. 2025. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*.

Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, brian ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, and 16 others. 2025. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 17–40. PMLR.

Blai Bonet, Guillem Francès, and Hector Geffner. 2019. Learning features and abstract actions for computing generalized plans. In *AAAI*, pages 2703–2710. AAAI Press.

Blai Bonet and Hector Geffner. 2021. General policies, representations, and planning width. In *AAAI*, pages 11764–11773. AAAI Press.

Blai Bonet and Hector Geffner. 2024. General policies, subgoal structure, and planning width. *Journal of Artificial Intelligence Research*, 80:475–516.

Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo De Lazcano Perez-Vicente, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and J K Terry. 2023. *Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks*. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

Li Dong and Mirella Lapata. 2018. *Coarse-to-fine decoding for neural semantic parsing*. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 731–742, Melbourne, Australia. Association for Computational Linguistics.

Dominik Drexler, Jendrik Seipp, and Hector Geffner. 2024. Expressing and exploiting subgoal structure in classical planning using sketches. *J. Artif. Intell. Res.*, 80.

Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, Jinlan Fu, Jingjing Gong, and Xipeng Qiu. 2025. *Libero-plus: In-depth robustness analysis of vision-language-action models*. *Preprint*, arXiv:2510.13626.

Chongkai Gao, Zixuan Liu, Zhenghao Chi, Junshan Huang, Xin Fei, Yiwen Hou, Yuxuan Zhang, Yudi Lin, Zhirui Fang, and Lin Shao. 2025. *VLA-OS: Structuring and dissecting planning representations and paradigms in vision-language-action models*. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Catherine Glossop, William Chen, Arjun Bhorkar, Dhruv Shah, and Sergey Levine. 2025. *Cast: Counterfactual labels improve instruction following in vision-language-action models*. *Preprint*, arXiv:2508.13446.

Danijar Hafner. 2021. Benchmarking the spectrum of agent capabilities. *arXiv preprint arXiv:2109.06780*.

Patrik Haslum, Nir Lipovetzky, Daniele Magazzeni, Christian Muise, Ronald Brachman, Francesca Rossi, and Peter Stone. 2019. *An introduction to the planning domain definition language*, volume 13. Springer.

Chi-Pin Huang, Yueh-Hua Wu, Min-Hung Chen, Yu-Chiang Frank Wang, and Fu-En Yang. 2025a. *Thinkact: Vision-language-action reasoning via reinforced visual latent planning*. *Preprint*, arXiv:2507.16815.

Hui Huang, Jiaheng Liu, Yancheng He, Shilong Li, Bing Xu, Conghui Zhu, Muyun Yang, and Tiejun Zhao. 2025b. *Musc: Improving complex instruction following with multi-granularity self-contrastive training*. In *ACL (1)*, pages 10667–10686. Association for Computational Linguistics.

Jiangyong Huang, Silong Yong, Xiaojian Ma, Xiongkun Linghu, Puhao Li, Yan Wang, Qing Li, Song-Chun

997	Zhu, Baoxiong Jia, and Siyuan Huang. 2024. An embodied generalist agent in 3d world. In <i>ICML</i> .	1054
998		1055
999	Emily Jin, Jiaheng Hu, Zhuoyi Huang, Ruohan Zhang, Jiajun Wu, Li Fei-Fei, and Roberto Martín-Martín. 2023. Mini-BEHAVIOR: A procedurally generated benchmark for long-horizon decision-making in embodied AI . In <i>NeurIPS 2023 Workshop on Generalization in Planning</i> .	1056
1000		1057
1001		1058
1002		1059
1003		1060
1004		1061
1005	Siddharth Karamcheti, Suraj Nair, Ashwin Balakrishna, Percy Liang, Thomas Kollar, and Dorsa Sadigh. 2024. Prismatic vlms: Investigating the design space of visually-conditioned language models . In <i>Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024</i> .	1062
1006		1063
1007		1064
1008		1065
1009		1066
1010		1067
1011	Jeonghye Kim, Sojeong Rhee, Minbeom Kim, Do-hyung Kim, Sangmook Lee, Youngchul Sung, and Kyomin Jung. 2025a. ReflAct: World-grounded decision making in LLM agents via goal-state reflection . In <i>Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing</i> , pages 33433–33465, Suzhou, China. Association for Computational Linguistics.	1068
1012		1069
1013		1070
1014		1071
1015		1072
1016		1073
1017		1074
1018		1075
1019	Moo Jin Kim, Chelsea Finn, and Percy Liang. 2025b. Fine-tuning vision-language-action models: Optimizing speed and success. <i>arXiv preprint arXiv:2502.19645</i> .	1076
1020		1077
1021		1078
1022		1079
1023	Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Paul Foster, Pannag R. Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. 2024. Openvla: An open-source vision-language-action model. In <i>CoRL</i> , volume 270 of <i>Proceedings of Machine Learning Research</i> , pages 2679–2713. PMLR.	1080
1024		1081
1025		1082
1026		1083
1027		1084
1028		1085
1029		1086
1030		1087
1031		1088
1032	Royi Lachmy, Valentina Pyatkin, Avshalom Manevich, and Reut Tsarfaty. 2022. Draw me a flower: Processing and grounding abstraction in natural language. <i>Trans. Assoc. Comput. Linguistics</i> , 10:1341–1356.	1089
1033		1090
1034		1091
1035		1092
1036	Chengshu Li, Fei Xia, Roberto Martín-Martín, Michael Lingelbach, Sanjana Srivastava, Bokui Shen, Kent Eliott Vainio, Cem Gokmen, Gokul Dharan, Tanish Jain, Andrey Kurenkov, Karen Liu, Hyowon Gweon, Jiajun Wu, Li Fei-Fei, and Silvio Savarese. 2022a. igibson 2.0: Object-centric simulation for robot learning of everyday household tasks . In <i>Proceedings of the 5th Conference on Robot Learning</i> , volume 164 of <i>Proceedings of Machine Learning Research</i> , pages 455–465. PMLR.	1093
1037		1094
1038		1095
1039		1096
1040		1097
1041		1098
1042		1099
1043		1100
1044		1101
1045		1102
1046	Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Michael Lingelbach, Jiankai Sun, Mona Anvari, Minjune Hwang, Manasi Sharma, Arman Aydin, Dhruva Bansal, Samuel Hunter, Kyu-Young Kim, Alan Lou, Caleb R. Matthews, and 9 others. 2022b. BEHAVIOR-1K: A benchmark for embodied AI with 1, 000 everyday activities and realistic simulation. In <i>CoRL</i> , volume 205 of <i>Proceedings of Machine Learning Research</i> , pages 80–93. PMLR.	1103
1047		1104
1048		1105
1049		1106
1050		1107
1051		1108
1052		1109
1053		1110
	Wei Li, Renshan Zhang, Rui Shao, Jie He, and Liqiang Nie. 2025. Cogvla: Cognition-aligned vision-language-action model via instruction-driven routing & sparsification. <i>Advances in neural information processing systems</i> .	1110
	Xuanlin Li, Kyle Hsu, Jiayuan Gu, Oier Mees, Karl Pertsch, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, Sergey Levine, Jiajun Wu, Chelsea Finn, Hao Su, Quan Vuong, and Ted Xiao. 2024. Evaluating real-world robot manipulation policies in simulation. In <i>CoRL</i> , volume 270 of <i>Proceedings of Machine Learning Research</i> , pages 3705–3728. PMLR.	1110
	Zechang Li, Yuxuan Lai, Yansong Feng, and Dongyan Zhao. 2020. Domain adaptation for semantic parsing . In <i>Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020</i> , pages 3723–3729. ijcai.org.	1110
	Zhixuan Liang, Yizhuo Li, Tianshuo Yang, Chengyue Wu, Sitong Mao, Liua Pei, Xiaokang Yang, Jiangmiao Pang, Yao Mu, and Ping Luo. 2025. Discrete diffusion vla: Bringing discrete diffusion to action decoding in vision-language-action policies. <i>arXiv preprint arXiv:2508.20072</i> .	1110
	Nir Lipovetzky and Hector Geffner. 2012. Width and serialization of classical planning problems. In <i>ECAI</i> , volume 242 of <i>Frontiers in Artificial Intelligence and Applications</i> , pages 540–545. IOS Press.	1110
	Nir Lipovetzky and Hector Geffner. 2017. Best-first width search: Exploration and exploitation in classical planning. In <i>AAAI</i> , pages 3590–3596. AAAI Press.	1110
	Yueen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. 2025. A survey on vision-language-action models for embodied ai . <i>Preprint</i> , arXiv:2405.14093.	1110
	NVIDIA, :, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi "Jim" Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, and 24 others. 2025. Gr00t n1: An open foundation model for generalist humanoid robots . <i>Preprint</i> , arXiv:2503.14734.	1110
	Abby O'Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, Albert Tung, Alex Bewley, Alexander Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai, Anchit Gupta, Andrew E. Wang, Anikait Singh, and 260 others. 2024. Open x-embodiment: Robotic learning datasets and RT-X models : Open x-embodiment collaboration. In <i>ICRA</i> , pages 6892–6903. IEEE.	1110

1111	Maxime Oquab, Timothée Darcet, Théo Moutakanni,	Gerald Jay Sussman. 1975. <i>A computer model of skill</i>	1169
1112	Huy V. Vo, Marc Szafraniec, Vasil Khalidov,	<i>acquisition</i> . Elsevier Science Inc.	1170
1113	Pierre Fernandez, Daniel Haziza, Francisco Massa,		
1114	Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas,	Hongyu Wang, Chuyan Xiong, Ruiping Wang, and Xilin	1171
1115	Wojciech Galuba, Russell Howes, Po-Yao Huang,	Chen. 2025. Bitvla: 1-bit vision-language-action	1172
1116	Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu	models for robotics manipulation. <i>arXiv preprint</i>	1173
1117	Sharma, and 7 others. 2024. Dinov2: Learning robust	<i>arXiv:2506.07530</i> .	1174
1118	visual features without supervision. <i>Trans. Mach.</i>		
1119	<i>Learn. Res.</i> , 2024.	Jeannette M. Wing. 2010. <i>Computational thinking:</i>	1175
		<i>What and why?</i> <i>The Link</i> .	1176
1120	Shishir G. Patil, Tianjun Zhang, Xin Wang, and		
1121	Joseph E. Gonzalez. 2024. Gorilla: Large language	Wei Wu, Fan Lu, Yunnan Wang, Shuai Yang, Shi	1177
1122	model connected with massive apis. In <i>NeurIPS</i> .	Liu, Fangjing Wang, Qian Zhu, He Sun, Yong	1178
		Wang, Shuailei Ma, Yiyu Ren, Kejia Zhang, Hui	1179
1123	Xavier Puig, Tianmin Shu, Shuang Li, Zilin Wang,	Yu, Jingmei Zhao, Shuai Zhou, Zhenqi Qiu, Hou-	1180
1124	Joshua B. Tenenbaum, Sanja Fidler, and Antonio	long Xiong, Ziyu Wang, Zechen Wang, and 6 others.	1181
1125	Torrallba. 2020. <i>Watch-and-help: A challenge for so-</i>	2026. <i>A pragmatic vla foundation model</i> . <i>Preprint</i> ,	1182
1126	<i>cial perception and human-ai collaboration</i> . <i>Preprint</i> ,	<i>arXiv:2601.18692</i> .	1183
1127	<i>arXiv:2010.09890</i> .		
1128	Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan	Lei Xiao, Jifeng Li, Juntao Gao, Feiyang Ye, Yan Jin,	1184
1129	Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang,	Jingjing Qian, Jing Zhang, Yong Wu, and Xiaoyuan	1185
1130	Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian,	Yu. 2025. Ava-vla: Improving vision-language-	1186
1131	Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li,	action models with active visual attention. <i>arXiv</i>	1187
1132	Zhiyuan Liu, and Maosong Sun. 2024. Toolllm: Fa-	<i>preprint arXiv:2511.18960</i> .	1188
1133	cilitating large language models to master 16000+		
1134	real-world apis. In <i>ICLR</i> . OpenReview.net.	Youguang Xing, Xu Luo, Junlin Xie, Lianli Gao,	1189
		Heng Tao Shen, and Jingkuan Song. 2025. <i>Short-</i>	1190
1135	Scott E. Reed, Konrad Zolna, Emilio Parisotto,	<i>cut learning in generalist robot policies: The role of</i>	1191
1136	Sergio Gómez Colmenarejo, Alexander Novikov,	<i>dataset diversity and fragmentation</i> . In <i>9th Annual</i>	1192
1137	Gabriel Barth-Maron, Mai Gimenez, Yuri Sulsky,	<i>Conference on Robot Learning</i> .	1193
1138	Jackie Kay, Jost Tobias Springenberg, Tom Eccles,		
1139	Jake Bruce, Ali Razavi, Ashley Edwards, Nicolas	Weimin Xiong, Yifan Song, Qingxiu Dong, Bingchan	1194
1140	Heess, Yutian Chen, Raia Hadsell, Oriol Vinyals,	Zhao, Feifan Song, XWang, and Sujian Li. 2025.	1195
1141	Mahyar Bordbar, and Nando de Freitas. 2022. A	<i>MPO: Boosting LLM agents with meta plan opti-</i>	1196
1142	generalist agent. <i>Trans. Mach. Learn. Res.</i> , 2022.	<i>mization</i> . In <i>Findings of the Association for Com-</i>	1197
		<i>putational Linguistics: EMNLP 2025</i> , pages 3914–	1198
1143	Lucy Xiaoyang Shi, Brian Ichter, Michael Robert Equi,	3935, Suzhou, China. Association for Computational	1199
1144	Liyiming Ke, Karl Pertsch, Quan Vuong, James Tan-	Linguistics.	1200
1145	ner, Anna Walling, Haohuan Wang, Niccolo Fusai,		
1146	Adrian Li-Bell, Danny Driess, Lachy Groom, Sergey	Ruochu Yang, Fumin Zhang, and Mengxue Hou. 2024.	1201
1147	Levine, and Chelsea Finn. 2025. Hi robot: Open-	<i>Oceanplan: Hierarchical planning and replanning</i>	1202
1148	ended instruction following with hierarchical vision-	for natural language AUV piloting in large-scale un-	1203
1149	language-action models. In <i>ICML</i> .	explored ocean environments. In <i>WUWNet</i> , pages	1204
		11:1–11:5. ACM.	1205
1150	Mohit Shridhar, Jesse Thomason, Daniel Gordon,		
1151	Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke	Shuai Yang, Hao Li, Yilun Chen, Bin Wang, Yang Tian,	1206
1152	Zettlemoyer, and Dieter Fox. 2020. Alfred: A bench-	Tai Wang, Hanqing Wang, Feng Zhao, Yiyi Liao, and	1207
1153	mark for interpreting grounded instructions for ev-	Jiangmiao Pang. 2025. <i>Instructvla: Vision-language-</i>	1208
1154	eryday tasks. In <i>Proceedings of the IEEE/CVF con-</i>	<i>action instruction tuning from understanding to ma-</i>	1209
1155	<i>ference on computer vision and pattern recognition</i> ,	<i>nipulation</i> . <i>Preprint</i> , <i>arXiv:2507.17520</i> .	1210
1156	pages 10740–10749.		
1157	Mustafa Shukor, Dana Aubakirova, Francesco Capuano,	Angen Ye, Zeyu Zhang, Boyuan Wang, Xiaofeng Wang,	1211
1158	Pepijn Kooijmans, Steven Palma, Adil Zouitine,	Dapeng Zhang, and Zheng Zhu. 2025. <i>Vla-r1: En-</i>	1212
1159	Michel Aractingi, Caroline Pascal, Martino Russi,	<i>hancing reasoning in vision-language-action models</i> .	1213
1160	Andres Marafioti, and 1 others. 2025. Smolvla: A	<i>Preprint</i> , <i>arXiv:2510.01623</i> .	1214
1161	vision-language-action model for affordable and effi-		
1162	cient robotics. <i>arXiv preprint arXiv:2506.01844</i> .	Cheng Yin, Yankai Lin, Wang Xu, Sikyuen Tam,	1215
		Xiangrui Zeng, Zhiyuan Liu, and Zhouping Yin.	1216
1163	Wenxuan Song, Jiayi Chen, Pengxiang Ding, Han Zhao,	2025. <i>Deepthinkvla: Enhancing reasoning capabil-</i>	1217
1164	Wei Zhao, Zhide Zhong, Zongyuan Ge, Zhijun Li,	ity of vision-language-action models. <i>arXiv preprint</i>	1218
1165	Donglin Wang, Lujia Wang, Jun Ma, and Haoang Li.	<i>arXiv:2511.15669</i> .	1219
1166	2025. PD-VLA: accelerating vision-language-action		
1167	model integrated with action chunking via parallel	Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov,	1220
1168	decoding. In <i>IROS</i> , pages 13162–13169. IEEE.	and Lucas Beyer. 2023. Sigmoid loss for language	1221
		image pre-training. In <i>ICCV</i> , pages 11941–11952.	1222
		IEEE.	1223

Ninghao Zhang, Bin Zhu, Shijie Zhou, and Jingjing Chen. 2026. [Restoring linguistic grounding in vla models via train-free attention recalibration](#). *Preprint*, arXiv:2603.06001.

Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, Ankur Handa, Tsung-Yi Lin, Gordon Wetzstein, Ming-Yu Liu, and Donglai Xiang. 2025. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. In *CVPR*, pages 1702–1713. Computer Vision Foundation / IEEE.

Huaixiu Steven Zheng, Swaroop Mishra, Xinyun Chen, Heng-Tze Cheng, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2024. Take a step back: Evoking reasoning via abstraction in large language models. In *ICLR*.

Yifan Zhong, Fengshuo Bai, Shaofei Cai, Xuchuan Huang, Zhang Chen, Xiaowei Zhang, Yuanfei Wang, Shaoyang Guo, Tianrui Guan, Ka Nam Lui, and 1 others. 2025. A survey on vision-language-action models: An action tokenization perspective. *arXiv preprint arXiv:2507.01925*.

Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong T. Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, and 35 others. 2023. RT-2: vision-language-action models transfer web knowledge to robotic control. In *CoRL*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR.

A Use of LLM Statement

An LLM was used only for polishing language; the scientific content is entirely original.

B Technical Appendix

Appendix Table of Contents

1. **Anticipatory defense on the choice of test environments and metrics (§ C)**
Justification for using MINI-BEHAVIOR over alternatives (e.g., iGibson, CALVIN) based on controllability, symbolic abstraction, and isolation of language-grounding challenges.
2. **Symbolic Representation of the Embodied Task (§ D)** Formal definition of states, actions, and planning problems using classical STRIPS/PDDL semantics.
3. **Details of the MINI-BEHAVIOR Environment (§ E)** Overview of scene layout, object types, action space, and perception model.

4. **Details of the MINI-BEHAVIOR-GRAN Extension (§ F)** Procedure for generating multi-granularity instructions, details on the feature set, oracle instructor, and dataset construction.

5. **Rule Set and Natural Language Prompt Examples (§ G)** Details of the rule set bank of 20 tasks and sample generated instructions at different granularities.

6. **Implementation Details (§ H)** Model architecture (Prismatic-7B backbone, SigLIP+DINOv2 visual encoders), training hyperparameters, optimizer settings. VLM Predictor architecture and training details.

7. **Additional Experimental Results (§ I)** Extended analysis and ablations.

C Anticipatory defense on the choice of test environments and metrics

In this work, we selected Mini-BEHAVIOR as our testbed. This choice is motivated by several key considerations aligned with our research objectives. Our research objective is not to solve the full stack of robotic embodiment (which includes perception, continuous control, and physics), but specifically to isolate and analyze the impact of instruction granularity on language grounding and long-horizon planning.

Therefore, high-fidelity simulators (e.g., iGibson (Li et al., 2022a), BEHAVIOR-1K (Li et al., 2022b)) introduce significant noise through complex texture rendering and continuous motor control. While realistic, these factors act as confounders. When an agent fails in such environments, it is often difficult to attribute the failure to a lack of linguistic understanding (granularity issues) or a failure in low-level execution (e.g., grasping physics or occlusion). Mini-BEHAVIOR retains the high-level decision complexity of household tasks—long horizons, multi-object interactions, and state dependencies—while abstracting away low-level sensorimotor noise.

In the perspective of fast prototyping and iterative experimentation, we have to also consider the rendering and physics simulation costs. Mini-BEHAVIOR’s grid-based discrete control and simplified visual representation allow for rapid data generation and model training.

Another consideration is the diversity of the task structures so that we can systematically vary instruction granularity across a wide range of sce-

narios. As such, Crafter (Hafner, 2021), which focuses on survival in a single domain, lacks the diverse task structures needed to obtain robust insights about instruction granularity.

Table 4 provides a detailed comparison of Mini-BEHAVIOR against other prominent benchmarks, highlighting why they were less suitable for this specific investigation.

D Symbolic Representation of the Embodied Task

We formalize the Mini-BEHAVIOR environment as a deterministic grid-based symbolic world. This allows us to represent the challenge as a classical planning problem $\mathcal{P} = \langle \mathcal{F}, \mathcal{A}, s_0, G \rangle$, where:

- $\mathcal{F}$ is a finite set of propositional fluents representing the state variables of the environment (e.g., object locations, states like *cooked* or *open*).
- $\mathcal{A}$ is a finite set of actions available to the agent.
- $s_0 \subseteq \mathcal{F}$ is the initial state configuration.
- $G \subseteq \mathcal{F}$ is the goal condition that must be satisfied.

A state $s \in \mathcal{S}$ is defined as a truth valuation over $\mathcal{F}$, with the state space $\mathcal{S} \subseteq 2^{\mathcal{F}}$ encompassing all valid combinations of fluent truth values.

E Details of MINI-BEHAVIOR Environment

To operationalize our formalism, we developed a **Universal PDDL Domain Model** (available at [minibehavior_gran_domain_model.pddl](#) in the codebase) capable of supporting all 20 household tasks without modification. This unified model comprises over 1,000 lines of PDDL 2.1 definitions and is fully compatible with the LAMA planner. We highly recommend checking the codebase for the complete domain model.

Table 5: Planning complexity in typical tasks.

Metric	Typical Range
Objects	15–40
Grid Locations	20–60
Ground Actions	10^3 – 10^5
Plan Length	50–200 primitive actions
Planning Time (LAMA)	0.5–300+ seconds

Search Space Characteristics

F Details of MINI-BEHAVIOR-GRAN Extension

MINI-BEHAVIOR provides the foundational task skeletons, which we extend in MINI-BEHAVIOR-GRAN to support multi-granularity instruction generation. A core component of this extension is the definition of a symbolic feature set used to generate rule-based policy sketches.

F.1 Feature Categories

The environment defines two primary categories of features: Boolean State Features and Quantitative Features.

F.1.1 Boolean State Features

These features evaluate whether a specific object instance satisfies a predicate. They require grounding (binding to specific object instances, e.g., *rag-01*).

F.1.2 Quantitative and Spatial Features

These features capture distances, counts, and spatial relationships. Crucially, they support temporal comparison allowing conditions such as “distance decreased” or “count dropped to zero.”

- **DistanceToNearest(type, condition):** Measures the Manhattan distance from the agent to the nearest object of a specific type.

– *Conditions:* $=0$ (adjacent), >0 , increase, decrease, change.

- **Count(type, condition, function):** Counts the number of objects of a specific type that satisfy a filter function.

– *Conditions:* $=0$, >0 , increase, decrease.

F.2 Count Functions

A diverse set of counting functions is implemented to support complex logical conditions (e.g., “all shoes are clean” is equivalent to *count_not_cleaned == 0*).

F.3 Reference Plan Generation and Task Complexity Analysis

While the original MINI-BEHAVIOR framework provides the high-level skeletons for household activities, it does not provide reference plans by default. Thus, we utilize the BFWS (Best-First Width Search) classical planner to generate reference plans for all 20 tasks in MINI-BEHAVIOR-GRAN. These reference plans serve as the ground

Table 4: Comparison of Embodied AI Environments regarding their suitability for studying Instruction Granularity.

Environment	Control	Horizon	Visuals	Symbolic Abstraction	Limitation
Mini-BEHAVIOR(*)	Discrete	Long	Grid	Support	Ideal balance: Supports symbolic state abstraction; diverse task suite; removes control noise while retaining logic complexity.
Crafter	Discrete	Long	Cartoon	Partial	Single task suite (survival); partially observable (memory heavy); few object types.
Meta-World	Continuous	Short	3D	No	Continuous control; no symbolic abstraction; focus on multi-task RL/manipulation, not long-horizon planning.
LIBERO	Continuous	Short	3D	No	Robotic arm focus; continuous control; no symbolic abstraction; VLA success rate already saturated (tasks relatively easy).
RoboTwin 2.0	Continuous	Short	3D	No	Continuous control; no symbolic abstraction; dual-arm manipulation focus; hard to segment expert trajectories and construct granular instructions.
CALVIN	Continuous	Medium	3D	No	Hard to design instruction granularity due to continuous action space (though provides diverse language); confounds planning with control difficulties.
ALFRED	Discrete	Long	3D Photo	Support	3D partially observable; poor infrastructure support for VLA training; high rendering cost.
iGibson / BEHAVIOR-1K	Continuous	Long	3D Photo	Partial	Good for long-horizon tasks, but 3D realistic vision context becomes a confounder.

Table 6: Boolean State Features

Feature	Applicable Objects	Semantics
Holding	All Items	Is the agent holding the specified object?
isOpen	Cabinets, Drawers, Doors	Is the container/door open?
isToggled	Sinks, Stoves, Switches	Is the appliance turned on?
isSoaked	Rags, Sponges	Is the cleaning tool wet?
isCleaned	Shoes, Dishes, Floor	Is the object free of stains?
isDustFree	Furniture, Surfaces	Is the object free of dust?

Table 7: Available Count Functions for Rule Definitions

Category	Function Description
State-Based	count_not_cleaned: Objects that are currently stained. count_not_soaked: Cleaning tools that are dry. count_not_toggled: Appliances that are turned off. count_closed: Containers/doors that are closed. count_not_sliced: Food items that require slicing.
Spatial	count_not_on_floor: Objects not placed on the floor layer. count_isolated: Objects with no neighbors of the same type. count_not_inside_target: Objects not contained within a specific furniture. count_not_ontop: Objects not placed on top of a specific surface. count_not_near_target: Objects not within 1 step of a target type.
Task-Specific	count_incomplete_salad: Plates lacking necessary ingredients. count_not_complete_tables: Tables lacking specific setups (e.g., candles).

truth for our agents and allow us to quantitatively categorize the complexity of each task.

Detailed statistics regarding the specific rule sets used for instruction generation and the resulting natural language statistics are discussed in the next section.

G Rule Set and Natural Language Prompt Examples For Different Tasks

We present the width-0 rule set for 10 over 20 household tasks in the Mini-BEHAVIOR-Gran environment. For $w > 0$, please refer to the codebase at [mini-behavior-gran/mini_behavior_utils/policy_sketch/](https://github.com/mini-behavior-gran/mini_behavior_utils/policy_sketch/).

G.1 Task 1: Laying Wood Floors

Features:

- H : holding an isolated wood (plywood)
- p : distance to the nearest isolated wood
- t : distance to an arbitrary (non-held) wood we'll place next to
- n : number of isolated woods

$\{\neg H, p > 0, n > 0\} \mapsto \{p \downarrow, t?\}$; move close to isolated wood
 $\{\neg H, p = 0, n > 0\} \mapsto \{H\}$; pick up isolated wood when reachable
 $\{H, t > 0, n > 0\} \mapsto \{t \downarrow\}$; move to some wood location
 $\{H, n > 0, t = 0\} \mapsto \{H?, n \downarrow, p?\}$; drop the isolated wood next to other wood

G.2 Task 2: Preparing Salad

Features:

- H_n : holding a not-sliceable food

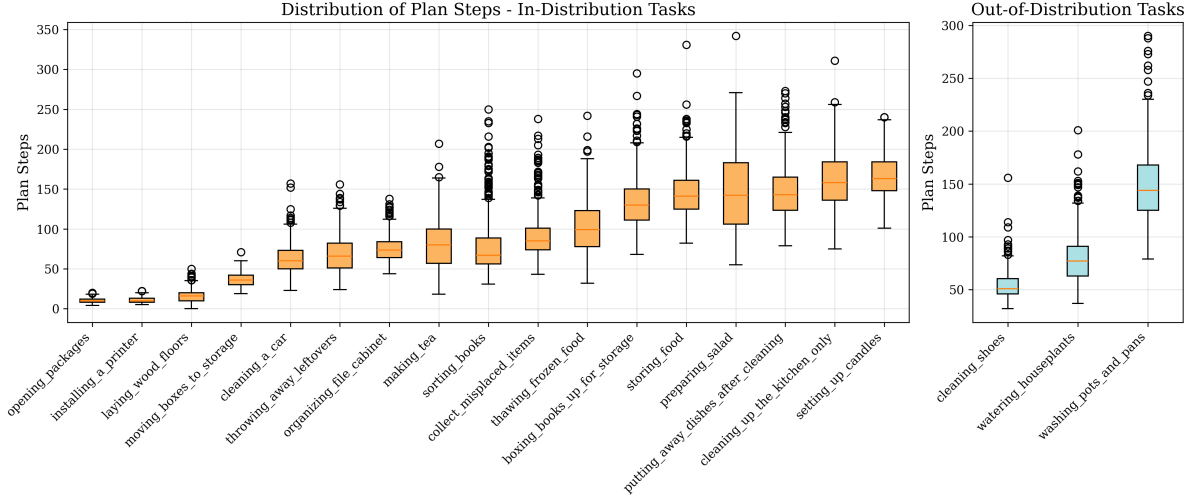


Figure 11: Distribution of reference plan steps required for tasks in MINI-BEHAVIOR-GRAN obtained by BWFS classical planner.

- H_s : holding a sliceable food
- H_k : holding a slicer
- U : number of sliceable food that is not sliced yet
- d_p : distance to the selected incomplete plate p
- d_n, d_s : distances to nearest not-sliceable / sliceable
- k : distance to a slicer (knife)
- B : selected plate p complete bottom salad
- n : number of incomplete plates
- nn : number of plates that do not have bottom salad yet

$\{U > 0, \neg H_k, k > 0\} \mapsto \{k \downarrow\}$; move to slicer
 $\{U > 0, \neg H_k, k = 0\} \mapsto \{H_k\}$; hold a slicer
 Acquire slicer: $\{U > 0, H_k, d_s > 0\} \mapsto \{d_s \downarrow\}$; move to sliceable food
 $\{U > 0, H_k, d_s = 0\} \mapsto \{U \downarrow, \neg H_k\}$; cut sliceable food
 $\{U = 0, H_k\} \mapsto \{\neg H_k\}$; put down slicer
 $\{n > 0, U = 0, nn > 0, \neg B, \neg H_n, d_n > 0\} \mapsto \{d_n \downarrow\}$
 Place bottom salad: $\{n > 0, U = 0, nn > 0, \neg B, \neg H_n, d_n = 0\} \mapsto \{H_n\}$
 $\{n > 0, U = 0, nn > 0, \neg B, H_n, d_p > 0\} \mapsto \{d_p \downarrow\}$
 $\{n > 0, U = 0, nn > 0, \neg B, H_n, d_p = 0\} \mapsto \{B, nn \downarrow, \neg H_n\}$
 $\{nn = 0, n > 0, U = 0, \neg H_s, d_s > 0\} \mapsto \{d_s \downarrow\}$; move to sliceable food
 Handle sliceable salad: $\{nn = 0, n > 0, U = 0, \neg H_s, d_s = 0\} \mapsto \{H_s\}$; pick up sliceable food
 $\{nn = 0, n > 0, U = 0, H_s, d_p > 0\} \mapsto \{d_p \downarrow\}$; move to the plate
 $\{nn = 0, n > 0, U = 0, H_s, d_p = 0\} \mapsto \{n \downarrow, \neg H_s\}$; put down sliced food

G.3 Task 3: Cleaning Up the Kitchen Only

Features:

- *Holding flags*: H_b (blender), H_f (food), H_s (soap), H_r (rag), H_p (plate), H_o (vegetable oil)
- *Distances*: d_{ct} (countertop), d_{fr} (fridge), d_{si} (sink), d_{c1} (cabinet for plate), d_{c2} (cabinet for oil), d_x (to object x)
- *State booleans*: $\text{Open}(c)$, $\text{Tog}(s)$, Soaked , $\text{DustFree}(c)$, $\text{Clean}(p)$, $\text{OilIn}(c_2)$, Diff

- *Counters*: N_b (blenders not on countertop), N_a (apples not in fridge), N_{cas} (casseroles not in fridge), N_s (soaps not next to sink), N_{cab} (cabinets not dusted), N_r (rags not next to sink), N_{p_clean} (plates not clean), N_{oil} (oil not in cabinet), N_{p_store} (plates not stored), N_{close} (closed furniture)

Open Cabinet and Fridge: $\{N_{close} > 0, d_c > 0\} \mapsto \{d_c \downarrow\}$; move toward closed furniture
 $\{N_{close} > 0, d_c = 0\} \mapsto \{N_{close} \downarrow\}$

$\{N_{close} = 0, \neg H_o, d_{oil} > 0\} \mapsto \{d_{oil} \downarrow\}$; approach oil
 $\{N_{close} = 0, \neg H_o, d_{oil} = 0\} \mapsto \{H_o\}$; pick up oil
 Oil placement: $\{N_{close} = 0, H_o, N_{oil} > 0, d_{c2} > 0\} \mapsto \{d_{c2} \downarrow\}$; go to cabinet c_2
 $\{N_{close} = 0, H_o, N_{oil} > 0, d_{c2} = 0\} \mapsto \{N_{oil} \downarrow, \neg H_o\}$; place oil in c_2

$\{\neg \text{Tog}(s), d_{si} > 0\} \mapsto \{d_{si} \downarrow\}$; toggle sink on
 $\{\neg \text{Tog}(s), d_{si} = 0\} \mapsto \{\text{Tog}(s)\}$
 $\{\neg H_r, d_{rag} > 0\} \mapsto \{d_{rag} \downarrow\}$; get rag
 $\{\neg H_r, d_{rag} = 0\} \mapsto \{H_r\}$
 Plate cleaning: $\{H_r, \neg \text{Soaked}, d_{si} > 0\} \mapsto \{d_{si} \downarrow\}$; soak rag
 $\{H_r, \neg \text{Soaked}, d_{si} = 0\} \mapsto \{\text{Soaked}\}$

$\{H_r, \text{Soaked}, N_{p_clean} > 0, d_{plate} > 0\} \mapsto \{d_{plate} \downarrow\}$
 $\{H_r, \text{Soaked}, N_{p_clean} > 0, d_{plate} = 0\} \mapsto \{N_{p_clean} \downarrow, d_{plate}?\}$
 $\{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, \neg H_p, N_{p_store} > 0, d_{plate} > 0\} \mapsto \{d_{plate} \downarrow\}$
 $\{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, \neg H_p, N_{p_store} > 0, d_{plate} = 0\} \mapsto \{H_p\}$
 $\{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, H_p, N_{p_store} > 0, d_{c1} > 0\} \mapsto \{d_{c1} \downarrow\}$
 $\{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, H_p, N_{p_store} > 0, d_{c1} = 0\} \mapsto \{N_{p_store} \downarrow, \neg H_p\}$

$\{N_b > 0, \neg H_b, d_{blender} > 0\} \mapsto \{d_{blender} \downarrow\}$
 $\{N_b > 0, \neg H_b, d_{blender} = 0\} \mapsto \{H_b\}$
 Blender to countertop: $\{H_b, N_b > 0, d_{ct} > 0\} \mapsto \{d_{ct} \downarrow\}$
 $\{H_b, N_b > 0, d_{ct} = 0\} \mapsto \{N_b \downarrow, \neg H_b\}$

$\{N_{close} = 0, \neg H_f, d_{apple} > 0\} \mapsto \{d_{apple} \downarrow\}$
 $\{N_{close} = 0, \neg H_f, d_{apple} = 0\} \mapsto \{H_f\}$
 Apple to fridge: $\{N_{close} = 0, H_f, N_a > 0, d_{fr} > 0\} \mapsto \{d_{fr} \downarrow\}$
 $\{N_{close} = 0, H_f, N_a > 0, d_{fr} = 0\} \mapsto \{N_a \downarrow, \neg H_f\}$

$\{N_{close} = 0, \neg H_f, d_{casserole} > 0\} \mapsto \{d_{casserole} \downarrow\}$
 $\{N_{close} = 0, \neg H_f, d_{casserole} = 0\} \mapsto \{H_f\}$
 Casserole to fridge: $\{N_{close} = 0, H_f, N_{cas} > 0, d_{fr} > 0\} \mapsto \{d_{fr} \downarrow\}$
 $\{N_{close} = 0, H_f, N_{cas} > 0, d_{fr} = 0\} \mapsto \{N_{cas} \downarrow, \neg H_f\}$

$\{\neg H_r, d_{rag} > 0\} \mapsto \{d_{rag} \downarrow\}$; get rag
 $\{\neg H_r, d_{rag} = 0\} \mapsto \{H_r\}$
 Cabinet dusting: $\{H_r, N_{cab} > 0, d_c > 0\} \mapsto \{d_c \downarrow\}$; clean cabinet
 $\{H_r, N_{cab} > 0, d_c = 0\} \mapsto \{N_{cab} \downarrow, \neg H_r\}$

Task	Split	Class	Goal Description	Mean Steps
opening_packages	ID	short	Open every package.	10.2
installing_a_printer	ID	short	Place a printer on a table and turn it on.	10.6
laying_wood_floors	ID	short	Lay every plywood sheet on the floor, each touching at least one other sheet.	15.9
moving_boxes_to_storage	ID	short	Place every carton inside a shelf.	36.3
cleaning_a_car	ID	short	Make at least one car dust-free. Place a rag and soap together inside a bucket.	62.6
throwing_away_leftovers	ID	middle	Throw away every hamburger by placing it in a trash can.	68.3
organizing_file_cabinet	ID	middle	Put a marker on a table and file every folder and document in a cabinet.	75.5
making_tea	ID	middle	Slice a lemon. Put a teapot on an activated stove. Keep a soaked teabag with the teapot.	78.7
sorting_books	ID	middle	Place every book and every hardback on a shelf.	78.8
collect_misplaced_items	ID	middle	Place all gym shoes, necklaces, notebooks, and socks on a table.	90.7
thawing_frozen_food	ID	middle	Place a date label next to a fish. Put every fish next to a sink. Set an olive next to a sink.	101.7
boxing_books_up_for_storage	ID	long	Place every book inside a box.	134.4
storing_food	ID	long	Store every cookable item inside a cabinet.	144.6
preparing_salad	ID	long	Slice all apples and tomatoes; place them on plates. Put every radish and lettuce on plates. Ensure each plate holds at least one cookable item.	146.7
putting_away_dishes_after_cleaning	ID	long	Put every plate inside a cabinet.	147.7
cleaning_up_the_kitchen_only	ID	long	Put every blender on a countertop. Refrigerate all apples and casseroles. Keep soap next to a sink. Make plates unstained and cabinets dust-free. Place each rag either beside or inside the sink. Store plates and vegetable oil in different cabinets.	161.0
setting_up_candles	ID	long	On every table, place three distinct candles on top.	165.2
cleaning_shoes	OOD	short	Leave a towel on the floor and make sure every shoe is unstained and dust-free.	54.2
watering_houseplants	OOD	middle	Water every potted plant until it is soaked.	79.4
washing_pots_and_pans	OOD	long	Clean every teapot, kettle, and pan so they're unstained, then store each inside a cabinet.	149.5

Table 8: Classification of tasks in MINI-BEHAVIOR-GRAN based on the median number of plan steps, as determined by the BWFS planner. Tasks are categorized into short, middle, and long classes to reflect varying planning complexity based on plan length.

1467	$\{N_{cab} = 0, N_{p_clean} = 0, \neg H_r, d_{rag} > 0\} \mapsto \{d_{rag} \downarrow\}$		$\{N_m > 0, \neg H_m, p_m > 0\} \mapsto \{p_m \downarrow, p_t?\}$	; move toward the nearest marker	1484
	$\{N_{cab} = 0, N_{p_clean} = 0, \neg H_r, d_{rag} = 0\} \mapsto \{H_r\}$		$\{N_m > 0, \neg H_m, p_m = 0\} \mapsto \{H_m, p_t?\}$	; pick up the marker when reachable	
	$\{N_{cab} = 0, N_{p_clean} = 0, H_r, N_r > 0, d_{si} > 0\} \mapsto \{d_{si} \downarrow\}$		$\{N_m > 0, H_m, p_t > 0\} \mapsto \{p_t \downarrow\}$	; move toward a table	
	$\{N_{cab} = 0, N_{p_clean} = 0, H_r, N_r > 0, d_{si} = 0\} \mapsto \{N_r \downarrow, \neg H_r\}$		$\{N_m > 0, H_m, p_t = 0\} \mapsto \{N_m \downarrow, \neg H_m, p_m?\}$	; put marker on table	
	$\{N_{cab} = 0, N_{p_clean} = 0, \neg H_s, d_{soap} > 0\} \mapsto \{d_{soap} \downarrow\}$		$\{N_f > 0, N_o = 0, \neg H_f, p_f > 0\} \mapsto \{p_f \downarrow, p_c?\}$		
1468	$\{N_{cab} = 0, N_{p_clean} = 0, \neg H_s, d_{soap} = 0\} \mapsto \{H_s\}$		$\{N_f > 0, N_o = 0, \neg H_f, p_f = 0\} \mapsto \{H_f, p_c?\}$		1485
	$\{N_{cab} = 0, N_{p_clean} = 0, H_s, N_s > 0, d_{si} > 0\} \mapsto \{d_{si} \downarrow\}$		$\{N_f > 0, N_o = 0, H_f, p_c > 0\} \mapsto \{p_c \downarrow\}$		
	$\{N_{cab} = 0, N_{p_clean} = 0, H_s, N_s > 0, d_{si} = 0\} \mapsto \{N_s \downarrow, \neg H_s\}$		$\{N_f > 0, N_o = 0, H_f, p_c = 0\} \mapsto \{N_f \downarrow, \neg H_f, p_f?\}$		
			$\{N_d > 0, N_o = 0, \neg H_d, p_d > 0\} \mapsto \{p_d \downarrow, p_c?\}$		
1469			$\{N_d > 0, N_o = 0, \neg H_d, p_d = 0\} \mapsto \{H_d, p_c?\}$		1486
1470			$\{N_d > 0, N_o = 0, H_d, p_c > 0\} \mapsto \{p_c \downarrow\}$		
			$\{N_d > 0, N_o = 0, H_d, p_c = 0\} \mapsto \{N_d \downarrow, \neg H_d, p_d?\}$		
1471					
1472					
1473					
1474					
1475					
1476					
1477					
1478					
1479					
1480					
1481					
1482					
1483					
	$\{N_o > 0, p_c > 0\} \mapsto \{p_c \downarrow\}$	; move toward a closed cabinet			
	$\{N_o > 0, p_c = 0\} \mapsto \{N_o \downarrow\}$	; open the cabinet when reachable			

G.4 Task 4: Organizing File Cabinet

Features:

- H_m : holding a marker
- H_f : holding a folder
- H_d : holding a document
- p_m : distance to the nearest marker
- p_f : distance to the nearest folder
- p_d : distance to the nearest document
- p_t : distance to a table
- p_c : distance to a valid cabinet target
- N_m : number of markers not on a table
- N_f : number of folders not in a cabinet
- N_d : number of documents not in a cabinet
- N_o : number of closed cabinets

G.5 Task 5: Thawing Frozen Food

Features:

- H_{fish} : holding a fish
- H_{olive} : holding an olive
- H_{date} : holding a date
- p_{fish} : distance to the nearest fish
- p_{olive} : distance to the nearest olive
- p_{date} : distance to the nearest date
- p_{sink} : distance to a sink
- N_{fish} : number of fish not next to a sink
- N_{olive} : number of olives not next to a sink
- N_{date} : number of dates not next to a fish

	$\{\neg H_{fish}, p_{fish} > 0\} \mapsto \{p_{fish} \downarrow\}$
1499	<i>Fish Task:</i> $\{\neg H_{fish}, p_{fish} = 0\} \mapsto \{H_{fish}\}$
	$\{H_{fish}, p_{sink} > 0\} \mapsto \{p_{sink} \downarrow\}$
	$\{H_{fish}, p_{sink} = 0\} \mapsto \{N_{fish} \downarrow, \neg H_{fish}, p_{fish}?\}$
	$\{\neg H_{olive}, p_{olive} > 0\} \mapsto \{p_{olive} \downarrow\}$
1500	<i>Olive Task:</i> $\{\neg H_{olive}, p_{olive} = 0\} \mapsto \{H_{olive}\}$
	$\{H_{olive}, p_{sink} > 0\} \mapsto \{p_{sink} \downarrow\}$
	$\{H_{olive}, p_{sink} = 0\} \mapsto \{N_{olive} \downarrow, \neg H_{olive}, p_{olive}?\}$
	$\{\neg H_{date}, N_{fish\ olive} = 0, p_{date} > 0\} \mapsto \{p_{date} \downarrow\}$
1501	<i>Date Task:</i> $\{\neg H_{date}, N_{fish\ olive} = 0, p_{date} = 0\} \mapsto \{H_{date}\}$
	$\{H_{date}, N_{fish\ olive} = 0, p_{fish} > 0\} \mapsto \{p_{fish} \downarrow\}$
	$\{H_{date}, N_{fish\ olive} = 0, p_{fish} = 0\} \mapsto \{N_{date} \downarrow, \neg H_{date}, p_{date}?\}$

G.6 Task 6: Making Tea

Features:

- H_l : holding a lemon
- H_{knife} : holding a knife
- H_t : holding a teapot
- H_{tb} : holding a tea bag
- p_l : distance to the nearest lemon
- p_{knife} : distance to the knife
- p_t : distance to the nearest teapot
- p_{tb} : distance to the nearest tea bag
- p_s : distance to the stove
- p_{sink} : distance to the sink
- `is_sliced(lemon)`: whether the lemon is sliced
- `is_toggled(stove)`: whether the stove is toggled on
- `is_toggled(sink)`: whether the sink is toggled on
- `is_soaked(tea_bag)`: whether the tea bag is soaked
- `atsamelocation(tea_bag, teapot)`: tea bag at same location as teapot
- `onTop(teapot, stove)`: teapot on stove

	$\{\neg \text{is_toggled}(\text{stove}), p_s > 0\} \mapsto \{p_s \downarrow\}$
1524	<i>Toggle stove:</i> $\{\neg \text{is_toggled}(\text{stove}), p_s = 0\} \mapsto \{\text{is_toggled}(\text{stove})\}$
	$\{\neg H_{knife}, p_{knife} > 0, \neg \text{is_sliced}(\text{lemon})\} \mapsto \{p_{knife} \downarrow\}$
	$\{\neg H_{knife}, p_{knife} = 0, \neg \text{is_sliced}(\text{lemon})\} \mapsto \{H_{knife}\}$
1525	<i>Slice lemon:</i> $\{H_{knife}, \neg \text{is_sliced}(\text{lemon}), p_l > 0\} \mapsto \{p_l \downarrow\}$
	$\{H_{knife}, \neg \text{is_sliced}(\text{lemon}), p_l = 0\} \mapsto \{\text{is_sliced}(\text{lemon})\}$
	$\{H_{knife}, \text{is_sliced}(\text{lemon})\} \mapsto \{\neg H_{knife}, p_{knife}?\}$
	$\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \neg \text{onTop}(\text{tp}, \text{st}), \neg H_t, p_t > 0\} \mapsto \{p_t \downarrow\}$
	$\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \neg \text{onTop}(\text{tp}, \text{st}), \neg H_t, p_t = 0\} \mapsto \{H_t\}$
	$\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \neg \text{onTop}(\text{tp}, \text{st}), H_t, p_s > 0\} \mapsto \{p_s \downarrow\}$
1526	<i>Handle teapot:</i> $\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \neg \text{onTop}(\text{tp}, \text{st}), H_t, p_s = 0\} \mapsto \{\text{onTop}(\text{tp}, \text{st}), \neg H_t, p_t?\}$
	$\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \text{onTop}(\text{tp}, \text{st}), \neg H_{tb}, p_{tb} > 0\} \mapsto \{p_{tb} \downarrow\}$
	$\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \text{onTop}(\text{tp}, \text{st}), \neg H_{tb}, p_{tb} = 0\} \mapsto \{H_{tb}\}$
	$\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \text{onTop}(\text{tp}, \text{st}), H_{tb}, p_t > 0\} \mapsto \{p_t \downarrow\}$
	$\{H_{tb}, \neg \text{atsameloc}(\text{tb}, \text{tp}), \text{onTop}(\text{tp}, \text{st}), p_t = 0\} \mapsto \{\text{atsameloc}(\text{tb}, \text{tp}), \neg H_{tb}, p_{tb}?\}$

G.7 Task 7: Opening Packages

Features:

- p : distance to the nearest package
 - N : number of unopened packages
- $\{N > 0, p > 0\} \mapsto \{p \downarrow\}$; move toward the nearest unopened package
 $\{N > 0, p = 0\} \mapsto \{N \downarrow\}$; open the package when reachable

G.8 Task 8: Boxing Books Up for Storage

Features:

- H : holding an unboxed book
 - p : distance to the nearest unboxed book
 - b : distance to a valid box target
 - N : number of unboxed books
- $\{\neg H, p > 0\} \mapsto \{p \downarrow, b?\}$; move toward the nearest unboxed book
 $\{\neg H, p = 0\} \mapsto \{H\}$; pick up the book when reachable
 $\{H, N > 0, b > 0\} \mapsto \{b \downarrow\}$; move toward a chosen box spot
 $\{H, N > 0, b = 0\} \mapsto \{\neg H, N \downarrow, p?\}$; place the book into the box

G.9 Task 9: Collect Misplaced Items

Features:

- H_s : holding a gym shoe
- H_n : holding a necklace
- H_b : holding a notebook
- H_k : holding a sock
- p_s : distance to the nearest gym shoe
- p_n : distance to the nearest necklace
- p_b : distance to the nearest notebook
- p_k : distance to the nearest sock
- p_t : distance to a table
- N_s : number of gym shoes not on a table
- N_n : number of necklaces not on a table
- N_b : number of notebooks not on a table
- N_k : number of socks not on a table

	$\{\neg H_s, N_s > 0, p_s > 0\} \mapsto \{p_s \downarrow, p_t?\}$
	$\{\neg H_s, N_s > 0, p_s = 0\} \mapsto \{H_s, p_t?\}$
1554	<i>Gym shoe task:</i> $\{H_s, N_s > 0, p_t > 0\} \mapsto \{p_t \downarrow\}$
	$\{H_s, N_s > 0, p_t = 0\} \mapsto \{N_s \downarrow, \neg H_s, p_s?\}$
	$\{\neg H_n, N_n > 0, p_n > 0\} \mapsto \{p_n \downarrow, p_t?\}$
	$\{\neg H_n, N_n > 0, p_n = 0\} \mapsto \{H_n, p_t?\}$
1555	<i>Necklace task:</i> $\{H_n, N_n > 0, p_t > 0\} \mapsto \{p_t \downarrow\}$
	$\{H_n, N_n > 0, p_t = 0\} \mapsto \{N_n \downarrow, \neg H_n, p_n?\}$
	$\{\neg H_b, N_b > 0, p_b > 0\} \mapsto \{p_b \downarrow, p_t?\}$
	$\{\neg H_b, N_b > 0, p_b = 0\} \mapsto \{H_b, p_t?\}$
1556	<i>Notebook task:</i> $\{H_b, N_b > 0, p_t > 0\} \mapsto \{p_t \downarrow\}$
	$\{H_b, N_b > 0, p_t = 0\} \mapsto \{N_b \downarrow, \neg H_b, p_b?\}$
	$\{\neg H_k, N_k > 0, p_k > 0\} \mapsto \{p_k \downarrow, p_t?\}$
	$\{\neg H_k, N_k > 0, p_k = 0\} \mapsto \{H_k, p_t?\}$
1557	<i>Sock task:</i> $\{H_k, N_k > 0, p_t > 0\} \mapsto \{p_t \downarrow\}$
	$\{H_k, N_k > 0, p_t = 0\} \mapsto \{N_k \downarrow, \neg H_k, p_k?\}$

G.10 Task 10: Putting Away Dishes After Cleaning

Features:

- H : holding a plate
- p : distance to the nearest plate
- b : distance to a valid cabinet target
- N : number of unboxed plates

- `is_opened`: whether the cabinet is opened

$\{\neg \text{is_opened}, b > 0\} \mapsto \{b \downarrow\}$; move toward the cabinet
 $\{\neg \text{is_opened}, b = 0\} \mapsto \{\text{is_opened}\}$; open the cabinet when reachable
 $\{\neg H, N > 0, p > 0, \text{is_opened}\} \mapsto \{p \downarrow, b?\}$
 $\{\neg H, N > 0, p = 0, \text{is_opened}\} \mapsto \{H, b?\}$
 $\{H, N > 0, b > 0, \text{is_opened}\} \mapsto \{b \downarrow\}$
 $\{H, N > 0, b = 0, \text{is_opened}\} \mapsto \{\neg H, N \downarrow, p?\}$

G.11 Instruction Generation Statistics

We used a template-based generator to translate the symbolic rule sets into natural language instructions, ensuring alignment with the VLA’s training data format. Table 9 presents the summary statistics for the generated instructions across the three granularity levels.

H Implementation Details

H.1 Model Architecture

Our policy is built upon the OpenVLA architecture (Kim et al., 2024). Specifically, we utilize the Prismatic-7B backbone (Karamcheti et al., 2024), which integrates a frozen Llama-2-7B language model with fused visual features. The visual encoder consists of a dual-stream setup combining SigLIP (for semantic understanding) and DINOv2 (for fine-grained spatial geometry) (Zhai et al., 2023; Oquab et al., 2024).

H.2 Training Configuration

We fine-tune the model using Low-Rank Adaptation (LoRA) on the language model backbone, keeping the visual encoders frozen. All training setting variants (Pure, Centroid, Axial) share the same underlying training hyperparameters to ensure fair comparison.

Hardware and Duration: Training was conducted on a single node equipped with $8 \times$ NVIDIA A100 GPUs. A standard training run for 100,000 steps took approximately 23 hours to complete.

Table 10 summarizes the core hyperparameters used across our experiments.

Table 10: Hyperparameters for OpenVLA fine-tuning on MINI-BEHAVIOR-GRAN.

Hyperparameter	Value
Base Model	Prismatic-7B (Llama-2)
Visual Encoders	SigLIP + DINOv2
Parameter Efficient Tuning	LoRA (Rank $r = 32$)
Compute Hardware	$8 \times$ NVIDIA A100
Training Duration	~ 23 hours
Optimization Steps	100,000
Steps Before Decay	80,000
Learning Rate	2.5×10^{-4}
Batch Size (per GPU)	7 – 8

We also train a lightweight VLM predictor to quantify instruction predictability via perplexity. The model is fine-tuned using Low-Rank Adaptation (LoRA) on the Qwen3-VL-4B-Instruct backbone, with vision encoders frozen to preserve visual grounding capabilities established during pretraining. All models are trained on MINI-BEHAVIOR-GRAN instruction and image training data pairs, with training and evaluation splits maintained consistently across experiments. **Hardware and Duration:** Training was conducted on a single node equipped with $4 \times$ NVIDIA RTX 5090 GPUs. Each training run for 2 epochs. Table 11 summarizes the core hyperparameters used for VLM predictor training.

Table 11: Hyperparameters for Qwen3-VL-4B-Instruct fine-tuning on MINI-BEHAVIOR-GRAN (High-domain).

Hyperparameter	Value
Base Model	Qwen3-VL-4B-Instruct
Visual Encoders	Frozen (Qwen3-VL native)
Parameter Efficient Tuning	LoRA (Rank $r = 64$, $\alpha = 128$)
LoRA Target Modules	All linear layers
Compute Hardware	$4 \times$ NVIDIA RTX 5090
Training Epochs	2
Learning Rate	1.0×10^{-5}
Learning Rate Scheduler	Cosine with 10% warmup
Batch Size (per GPU)	32
Seed	[42,50,60]
Gradient Accumulation Steps	1
DeepSpeed Strategy	Zero-2

I Additional Experimental Results

This section provides additional detailed per-task performance breakdowns, and failure mode distributions.

I.1 Detailed Task Performance

Figure 12 details the success rates across all 20 tasks in the benchmark. We see some tasks are

Table 9: Statistics of generated natural language instructions across granularity levels.

Granularity	Count	Avg Length (words)	Std Length	Min	Max	Unique Tokens	TTR (Type-Token Ratio)	Avg Sent. Length
Fine (F)	1636	41.12	11.36	19	87	140	0.0021	41.12
Medium (M)	859	38.62	12.76	22	81	108	0.0033	38.62
Coarse (C)	587	33.60	13.39	22	95	94	0.0048	33.60

too difficult to solve and thus low across all instruction granularity. And then we also see the U shape trends in most tasks, showing the same phenomenon we discussed in (Q1) that both very fine and very coarse instructions are easier for VLA to follow.

I.2 Failure Analysis

Figure 13 presents the distribution of failure types across varying instruction widths. It confirms that low width encounters regression failures more frequently, while high width predominantly leads to stagnation failures. This aligns with our earlier observations in (Q3) regarding the distinct failure modes associated with different instruction granularities.

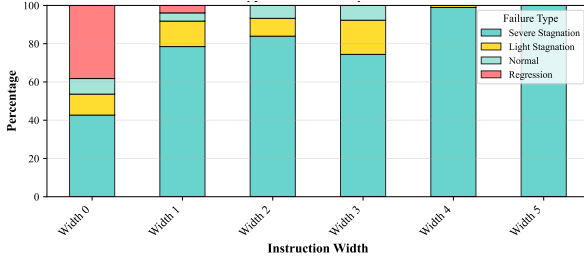


Figure 13: Failure type distribution across instruction widths.

I.3 More Plots for Width vs. Other Linguistic Heuristics as Metric for Language Grounding Complexity

As shown in the correlation heatmap Figure 14, w is the only metric that maintains a stable negative correlation with Success Rate across all horizons. In contrast, surface metrics like Token Count and Entity Count exhibit near-zero or even positive correlations in shorter horizons (e.g., $r = 0.09$ for 1-5 steps). This confirms the Complexity Paradox: while longer, more detailed instructions (higher token/entity counts) could actually provide helpful guidance that improves SR, increasing the latent planning width (w) invariably degrades performance.

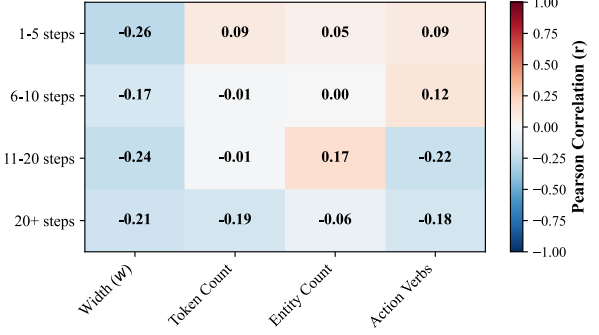


Figure 14: Correlation heatmap between instruction width and other linguistic heuristics.

Similarly, the consistency analysis in Figure 15 shows that w possesses the lowest standard deviation ($\sigma = 0.033$), indicating its predictive reliability is invariant to task horizons. Surface metrics, particularly Action Verbs ($\sigma = 0.155$), are highly volatile. It also shows that w provides the highest mean predictive power ($|r| = 0.220$), more than doubling the predictive strength of Token Count (0.076) or Entity Count (0.072).

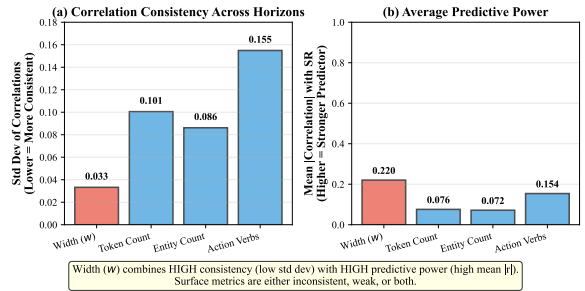


Figure 15: Consistency comparison between instruction width and other linguistic heuristics.

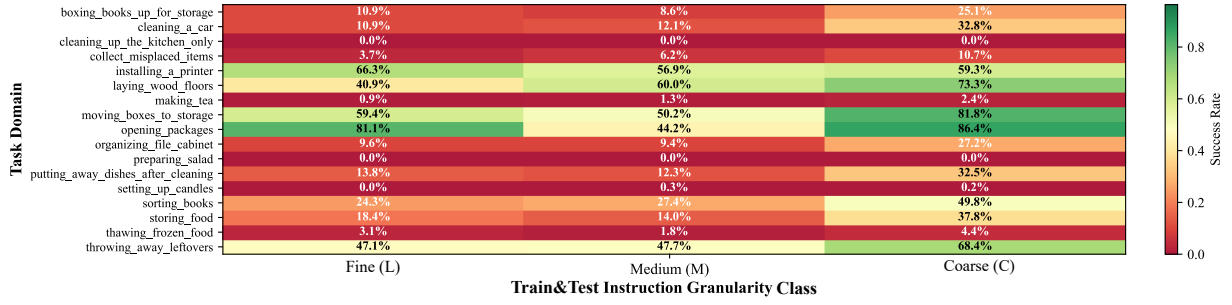


Figure 12: Detailed per-task success rates.

I.4 VLA Transformer Attention Weight Visualization

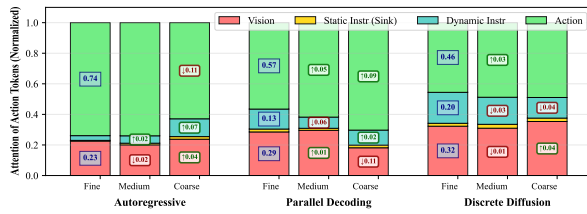


Figure 16: Attention from action tokens. Self-attention dominates, followed by vision; No clear trend across granularity levels.

Attention analysis (Figure 16) reveals there is no obvious reduction in language-token attention. This suggests that in VLA architectures, maintained attention weights do not necessarily equate to language grounding, highlighting a complex dynamic that requires future investigation.

Nevertheless, a closer inspection of the attention distributions reveals distinct architectural behaviors and compensatory mechanisms.

Autoregressive Models (Compensatory Reading). Autoregressive generation exhibits a heavy baseline reliance on historical action tokens (accounting for 0.74 of normalized attention in the Fine setting). Interestingly, when presented with Coarse instructions, attention to the dynamic instruction increases ($\uparrow 0.07$) while action attention decreases ($\downarrow 0.11$). This suggests a compensatory mechanism: as the rule width w increases and immediate subgoals disappear, the model attempts to ground more heavily on the language prompt to search for clues.

Parallel Decoding (Attending more on action modality). This architecture demonstrates a stark multimodal decoupling under coarse instructions. The attention paid to visual tokens drops significantly ($\downarrow 0.11$) in the Coarse setting, accompanied

by an increased reliance on generated action tokens ($\uparrow 0.09$). This could imply that without detailed instruction guidance at each step, parallel decoding intend to trust more on action history to decide the next actions, showing a different strategy from autoregressive models.

Discrete Diffusion (Structural Stability). Diffusion models exhibit the most balanced attention distribution, maintaining the highest baseline attention to both dynamic instructions (0.20) and visual features (0.32). Notably, this distribution remains remarkably rigid regardless of instruction granularity (with fluctuations bounded within ± 0.04).

I.5 Sample Generated Instructions by Granularity

To illustrate the qualitative differences between granularity levels, we provide five random samples for each category below. Note how Low (L) granularity explicitly specifies navigation and preconditions (e.g., “move toward,” “not yet at”), while High (H) granularity focuses almost exclusively on the final state change (e.g., “reduce the count of...”), shown in Table 12.

Fine Granularity (F) Samples

1. At step 11: When the count of uncleaned plate is above 0, not yet at sink 0 and sink is off, please try to move toward sink 0.
2. At step 8: When not yet at soap 0, not holding soap 0, the count of unwiped car is 0 and the count of rag not inside bucket is 0, please try to move toward soap 0.
3. At step 27: When the count of chip and oatmeal sugar and vegetable oil not inside cabinet is above 0, you are at oatmeal 1 and not holding oatmeal 1, please pick up oatmeal 1.
4. At step 15: When the count of plate not inside cabinet is above 0, you are at plate 3, the count of closed cabinet is 0 and not holding plate 3, please pick up plate 3.
5. At step 13: When the count of fish and olive not near sink is above 0, not yet at olive 0 and not holding olive 0, please try to move toward olive 0.

Medium Granularity (M) Samples

1. At step 2: When the count of plate not inside cabinet is above 0, the count of closed cabinet is 0 and not holding plate 2, please pick up plate 2.
2. At step 6: When the count of uncleaned plate is above 0, the count of dry rag is above 0 and holding rag 0, please reduce the count of dry rag.
3. At step 9: When the count of plate not inside cabinet 0 is above 0, the count of closed cabinet is 0, the count of vegetable oil not inside cabinet 1 is 0, the count of uncleaned plate is 0 and not holding plate 0, please pick up plate 0.
4. At step 13: When the count of book not inside box is above 0 and not holding book 2, please pick up book 2.
5. At step 9: When the count of chip and oatmeal sugar and vegetable oil not inside cabinet is above 0, holding chip 1 and the count of closed cabinet is 0, please not holding chip 1, then reduce the count of chip and oatmeal sugar and vegetable oil not inside cabinet.

Coarse Granularity (C) Samples

1. At step 2: When the count of hamburger not inside ashcan is above 0, please reduce the count of hamburger not inside ashcan.
2. At step 5: When the count of plate not inside cabinet is above 0 and the count of closed cabinet is 0, please reduce the count of plate not inside cabinet.
3. At step 3: When the count of teapot not on stove is above 0, please reduce the count of teapot not on stove.
4. At step 9: When the count of plate not inside cabinet is above 0 and the count of closed cabinet is 0, please reduce the count of plate not inside cabinet.
5. At step 8: When the count of chip and oatmeal sugar and vegetable oil not inside cabinet is above 0 and the count of closed cabinet is 0, please reduce the count of chip and oatmeal sugar and vegetable oil not inside cabinet.

Table 12: Sample generated instructions across granularity levels.